Appunti di Algorithm Engineering – Parte II

Appunti del Corso

Anno Accademico 2026/2027

Indice

1 Fondamenti di Compressione Dati e Teoria dell'Informazione 1
- 1.1 Modello Formale della Sorgente di Informazione 1
- 1.2 L'Entropia della Sorgente $H(S)$ 2
- 1.3 Codici Privi di Prefisso (Prefix-Free Codes) e Decodificabilità Univoca 4
- 1.4 Teorema di Shannon per la Compressione e Lunghezza Media 6
- 1.5 Analisi Comparativa: Efficienza ed Ottimalità di Codifica . . . 7
- 1.6 Rappresentazione ad Albero Binario e la Regola Aurea 9
- 1.7 Architettura Fisica del File Compresso e Processo di Decodifica 11
- 1.8 Modelli per la Stima delle Probabilità 13

2 Compressori Statistici, Codifica di Huffman e Forma Canonica 14
- 2.1 Introduzione ai Compressori Statistici e Modello Semi-Statico 14
- 2.2 Costruzione dell'Albero di Huffman: Approccio Greedy 15
- 2.3 Analisi di Ottimalità e Limiti Teorici di Efficienza 17
- 2.4 Teorema sull'Efficienza di Huffman ed Effetto della Distribuzione 19
- 2.5 Architettura del File Semi-Statico e Limiti degli Alberi Standard 21
- 2.6 Proprietà Strutturali e Combinatorie degli Alberi di Huffman . 22
- 2.7 Codifica Canonica di Huffman (Canonical Huffman) 23
- 2.8 Esempio Completo di Costruzione del Codice Canonico 24
- 2.9 Decodifica Senza Albero (Tree-less Decoding) 26
- 2.10 Valutazione Architetturale e Prospettive Avanzate 28

3 Codifica Aritmetica (*Arithmetic Coding*) 30
- 3.1 Principi Fondamentali e Procedura Converter 30
- 3.2 Lemma sull'Approssimazione Binaria e Troncamento 32
- 3.3 Processo di Compressione per Suddivisione Progressiva 33
- 3.4 Generazione dell'Output e Punto Medio 35
- 3.5 Efficienza della Codifica Aritmetica 38
- 3.6 Analisi Comparativa: Codifica Aritmetica vs Huffman 40

4 Esercitazioni di Compressione Statistica: Huffman e Codifica Aritmetica **42**
4.1 Introduzione e Struttura della Sessione di Esercizi 42
4.2 Esercizio 1: Codifica di Huffman Standard e Analisi Topologica 43
4.3 Esercizio 1: Costruzione della Codifica di Huffman Canonica . 46
4.4 Esercizio 1: Decodifica Senza Albero 49
4.5 Esercizio 2: Codifica Aritmetica – Modello di Sorgente e Partizioni 52
4.6 Esercizio 2: Decodifica Aritmetica Passo-Passo Mediante Zoom-In 53
4.7 Quadro Comparativo e Metodologico 57

5 Compressione Avanzata: Aritmetica Dinamica e Codici Ottimi **59**
5.1 Quadro Generale della Sessione di Esercizi 59
5.2 Codifica Aritmetica Adattiva a Modello Dinamico 60
5.3 Huffman Canonico con Distribuzione Sbilanciata 67
5.4 Codici Ottimi Non Derivabili da Huffman 70
5.5 Quadro Comparativo e Metodologico 74

Bibliografia **77**

Capitolo 1

Fondamenti di Compressione Dati e Teoria dell'Informazione

1.1 Modello Formale della Sorgente di Informazione

Il modello teorico di riferimento per la compressione dei dati affonda le proprie radici nel lavoro pionieristico di Claude Shannon del 1948 [1]. In questo paradigma fondazionale, il processo generativo dei dati viene descritto mediante un'entità astratta denominata **sorgente discreta di informazione**, convenzionalmente indicata con il simbolo S.

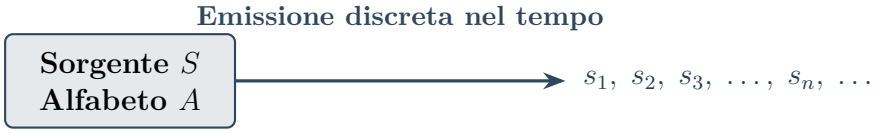


Figura 1.1: Modello formale della sorgente discreta di informazione: S emette una successione temporale discreta di simboli appartenenti all'alfabeto finito A.

1.1.1 Definizione di Alfabeto e Distribuzione di Probabilità

La sorgente S emette sequenzialmente una successione discreta di simboli nel tempo:

$$S \implies s_1,\ s_2,\ s_3,\ \ldots \tag{1.1}$$

Ciascun simbolo generato appartiene a un insieme finito, discreto e non vuoto denominato **alfabeto della sorgente**, indicato formalmente con A:

- La grandezza dell'alfabeto corrisponde alla sua cardinalità finita, denotata con $|A|$;
- Ciascun generico elemento appartenente all'alfabeto è indicato con la lettera σ, per cui $\sigma \in A$;
- Ad ogni simbolo $\sigma \in A$ è associata una probabilità a priori di emissione, formalizzata come la probabilità che il generico evento emesso coincida con σ:

$$p(\sigma) = \mathbb{P}(s_t = \sigma), \quad \forall \sigma \in A. \tag{1.2}$$

Trattandosi di una distribuzione di probabilità valida sullo spazio campionario discreto A, i valori di $p(\sigma)$ soddisfano rigidamente gli assiomi fondamentali di Kolmogorov:

$$\sum_{\sigma \in A} p(\sigma) = 1 \quad \text{e} \quad p(\sigma) \geq 0, \quad \forall \sigma \in A. \tag{1.3}$$

1.2 L'Entropia della Sorgente $H(S)$

L'**entropia** di una sorgente discreta, introdotta da Shannon [1], costituisce la misura quantitativa fondamentale della quantità media di informazione prodotta da S per ciascun simbolo emesso, nonché dell'incertezza intrinseca del processo generativo.

1.2.1 Definizione Matematica e Quantità di Sorpresa

Definizione 1.2.1 (Entropia di Shannon)**.** Data una sorgente discreta priva di memoria S caratterizzata da un alfabeto finito A e da una distribuzione di probabilità $p(\sigma)$, l'**entropia** $H(S)$ è definita come:

$$H(S) = \sum_{\sigma \in A} p(\sigma) \log_2 \frac{1}{p(\sigma)}. \tag{1.4}$$

Sfruttando le proprietà fondamentali dei logaritmi ($\log_2(x^{-1}) = -\log_2 x$), l'espressione (1.4) si riformula nella classica espressione canonica:

$$H(S) = -\sum_{\sigma \in A} p(\sigma) \log_2 p(\sigma). \tag{1.5}$$

Interpretazione Fisica e Unità di Misura. L'unità di misura dell'entropia, quando la base del logaritmo è pari a 2, è il **bit per simbolo** (indicato anche come *shannon*):

- Il termine $\log_2 \frac{1}{p(\sigma)}$ rappresenta la **self-information** (o quantità di sorpresa / informazione propria) apportata dal verificarsi dell'evento σ;
- Simboli rari ($p(\sigma) \to 0$) posseggono una self-information molto elevata ($\log_2 \frac{1}{p(\sigma)} \to \infty$), in quanto il loro verificarsi apporta una sorpresa notevole;
- Simboli frequenti ($p(\sigma) \to 1$) posseggono una self-information prossima a zero ($\log_2 1 = 0$), poiché il loro accadimento è ampiamente atteso e apporta scarsa informazione;
- L'entropia $H(S)$ rappresenta dunque la **speranza matematica** della self-information:

$$H(S) = \mathbb{E}\left[\log_2 \frac{1}{p(\sigma)}\right]. \tag{1.6}$$

1.2.2 Limiti di Estremo dell'Entropia

L'entropia di una qualunque sorgente discreta con alfabeto finito A è confinata all'interno di un intervallo chiuso ben determinato:

$$0 \leq H(S) \leq \log_2 |A|. \tag{1.7}$$

Proprietà 1.2.1 (Condizioni di Minimo e Massimo dell'Entropia)**.** *I valori limite dell'entropia corrispondono ai seguenti scenari stocastici estremi:*

1. ***Limite Inferiore*** *(**$H(S) = 0$**):* *si verifica se e solo se la sorgente è totalmente priva di incertezza. Ciò accade se esiste un simbolo deterministico $\sigma_k \in A$ con probabilità unitaria:*

$$p(\sigma_k) = 1 \quad e \quad p(\sigma) = 0, \quad \forall \sigma \neq \sigma_k. \tag{1.8}$$

   In questo scenario limite la sorgente emette deterministicamente sempre il medesimo simbolo: l'informazione trasmessa è nulla poiché non vi è alcuna incertezza da dissipare.

2. ***Limite Superiore*** *(**$H(S) = \log_2 |A|$**):* *si realizza se e solo se tutti i simboli dell'alfabeto possiedono la medesima probabilità di emissione, ovvero sotto una **distribuzione di probabilità uniforme**:*

$$p(\sigma) = \frac{1}{|A|}, \quad \forall \sigma \in A. \tag{1.9}$$

Dimostrazione del Massimo Uniforme. Sostituendo la distribuzione uniforme $p(\sigma) = \frac{1}{|A|}$ all'interno della definizione (1.4):

$$\begin{aligned} H(S) &= \sum_{\sigma \in A} \frac{1}{|A|} \log_2 \left(\frac{1}{\frac{1}{|A|}} \right) \\ &= \sum_{\sigma \in A} \frac{1}{|A|} \log_2 |A| \\ &= \log_2 |A| \sum_{\sigma \in A} \frac{1}{|A|} \\ &= \log_2 |A| \cdot 1 = \log_2 |A|. \end{aligned} \tag{1.10}$$

Poiché la funzione logaritmo è concava, la disuguaglianza di Jensen assicura che tale valore costituisce il massimo assoluto per qualsiasi distribuzione su $|A|$ elementi. □

1.2.3 Caso Notevole: Alfabeto Booleano

Nel caso specifico in cui l'alfabeto della sorgente sia di natura binaria o booleana, la cardinalità dell'insieme è pari a $|A| = 2$ (ad esempio $A = \{0, 1\}$):

- La capacità informativa massima teorica coincide con:

$$\log_2 |A| = \log_2 2 = 1 \text{ bit}; \tag{1.11}$$

- L'intervallo operativo dell'entropia per una sorgente booleana è pertanto ristretto alla relazione:

$$0 \leq H(S) \leq 1 \text{ bit per simbolo.} \tag{1.12}$$

1.3 Codici Privi di Prefisso (Prefix-Free Codes) e Decodificabilità Univoca

La trasmissione e l'archiviazione compressa dei dati generati dalla sorgente S richiedono un'operazione di mappatura biunivoca dei simboli dell'alfabeto in opportune sequenze binarie.

1.3.1 Definizione Formale di Codice Prefisso

Definizione 1.3.1 (Codice Privo di Prefisso / Prefix-Free Code)**.** Dato un alfabeto di sorgente A, uno schema di codifica binaria C assegna a ciascun

simbolo $\sigma \in A$ una sequenza finita di bit, denominata **parola di codice** (*codeword*), indicata con $c(\sigma)$. Il codice C si dice **privo di prefisso** (*prefix-free code*, spesso abbreviato per consuetudine come codice prefisso) se nessuna parola di codice valida costituisce il prefisso iniziale di un'altra parola di codice valida appartenente allo stesso codice:

$$\forall \sigma_i, \sigma_j \in A \text{ con } \sigma_i \neq \sigma_j \implies c(\sigma_i) \text{ non è prefisso di } c(\sigma_j). \tag{1.13}$$

Decodificabilità Univoca Istantanea. La proprietà di prefisso garantisce che il codice sia **univocamente decodificabile in modo istantaneo**:

- La scansione del flusso di bit può avvenire rigorosamente da sinistra verso destra (*online streaming*);
- Non appena una sottosequenza di bit ricevuta coincide con una parola di codice valida, il decodificatore riconosce con certezza il simbolo emesso, senza necessità di attendere i bit futuri, né di impiegare separatori speciali tra parole, né di ricorrere a costosi algoritmi di *backtracking*.

1.3.2 Controesempio Operativo: Violazione della Proprietà di Prefisso

Per cogliere l'essenzialità del vincolo (1.13), consideriamo un alfabeto ternario $A = \{a, b, c\}$ a cui viene applicato lo schema di codifica:

- $a \to 0$
- $b \to 1$
- $c \to 00$

Analisi dell'Ambiguità di Decodifica

In questo schema, la parola di codice $c(a) = 0$ è un prefisso proprio della parola $c(c) = 00$. Se il decodificatore riceve in ingresso il flusso binario:

$$\texttt{00}\ldots \tag{1.14}$$

si genera un'ambiguità irrisolvibile:

- La sequenza `00` può essere interpretata come la concatenazione di due simboli a ($c(a)c(a) = 00 \implies aa$);
- Oppure può rappresentare una singola emissione del simbolo c ($c(c) = 00 \implies c$).

Tale codice **non è privo di prefisso** e risulta **non univocamente decodificabile** senza informazione contestuale ausiliaria.

1.4 Teorema di Shannon per la Compressione e Lunghezza Media

1.4.1 Definizione di Lunghezza Media di un Codice

Definizione 1.4.1 (Lunghezza Media del Codice)**.** Data una sorgente discreta S con distribuzione di probabilità $p(\sigma)$ e uno schema di codifica privo di prefisso C che mappa ogni $\sigma \in A$ nella parola $c(\sigma)$, si definisce **lunghezza media del codice** $L(C)$ (spesso indicata con $\bar{L}$) la speranza matematica della lunghezza in bit delle parole di codice:

$$L(C) = \sum_{\sigma \in A} p(\sigma) \cdot |c(\sigma)|, \tag{1.15}$$

ove $|c(\sigma)|$ denota il numero intero di bit che costituiscono la codeword $c(\sigma)$.

1.4.2 Enunciato del Teorema Fondamentale di Shannon (1948)

Teorema 1.4.1 (Teorema di Shannon per la Codifica di Sorgente)**.** *Sia S una sorgente discreta con entropia $H(S)$. Per qualsiasi codice privo di prefisso C definito sull'alfabeto di S, la lunghezza media del codice $L(C)$ soddisfa la disuguaglianza:*

$$L(C) \geq H(S). \tag{1.16}$$

L'entropia di Shannon funge dunque da **limite inferiore invalicabile** per qualunque algoritmo di compressione senza perdita (*lossless*): nessun algoritmo di codifica a simboli singoli può rappresentare in media la sorgente impiegando un numero di bit inferiore a $H(S)$.

1.5 Analisi Comparativa: Efficienza ed Ottimalità di Codifica

Per illustrare il divario tra un codice semplicemente univoco e un codice pienamente efficiente, consideriamo un esempio analitico su un alfabeto ternario $A = \{a, b, c\}$.

1.5.1 Calcolo dell'Entropia di Riferimento

Si consideri la seguente distribuzione di probabilità a priori:

$$p(a) = \frac{1}{2}, \quad p(b) = \frac{1}{4}, \quad p(c) = \frac{1}{4}. \tag{1.17}$$

Sviluppiamo esplicitamente il calcolo dell'entropia $H(S)$:

$$\begin{aligned} H(S) &= \sum_{\sigma \in \{a,b,c\}} p(\sigma) \log_2 \frac{1}{p(\sigma)} \\ &= p(a) \log_2 \left(\frac{1}{p(a)} \right) + p(b) \log_2 \left(\frac{1}{p(b)} \right) + p(c) \log_2 \left(\frac{1}{p(c)} \right) \\ &= \frac{1}{2} \log_2(2) + \frac{1}{4} \log_2(4) + \frac{1}{4} \log_2(4). \end{aligned} \tag{1.18}$$

Poiché $\log_2(2) = 1$ e $\log_2(4) = 2$, otteniamo:

$$H(S) = \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{4} \cdot 2 = \frac{1}{2} + \frac{1}{2} + \frac{1}{2} = \frac{3}{2} = 1.5 \text{ bit per simbolo.} \tag{1.19}$$

Il limite teorico imposto dalla fisica dell'informazione sancisce che nessun codice privo di prefisso potrà mai scendere sotto la soglia di 1.5 bit per simbolo.

1.5.2 Caso 1: Codice Univocamente Decodificabile ma Sub-Ottimo

Consideriamo il primo schema di codifica C_1:

- $a \to 0 \quad (|c(a)| = 1 \text{ bit})$
- $b \to 10 \quad (|c(b)| = 2 \text{ bit})$
- $c \to 110 \quad (|c(c)| = 3 \text{ bit})$

Questo codice è privo di prefisso (nessuna sequenza è prefisso di un'altra), dunque è univocamente decodificabile. Calcoliamone la lunghezza media:

$$\begin{aligned} L(C_1) &= \sum_{\sigma \in A} p(\sigma) \cdot |c(\sigma)| \\ &= p(a) \cdot 1 + p(b) \cdot 2 + p(c) \cdot 3 \\ &= \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{4} \cdot 3 \\ &= \frac{1}{2} + \frac{1}{2} + \frac{3}{4} = 1 + \frac{3}{4} = \frac{7}{4} = 1.75 \text{ bit per simbolo.} \end{aligned} \tag{1.20}$$

Verifica del Teorema di Shannon per C_1

La disuguaglianza di Shannon è pienamente rispettata:

$$L(C_1) = 1.75 \geq H(S) = 1.5 \text{ bit.} \tag{1.21}$$

Tuttavia, il codice presenta una **ridondanza** non nulla pari a:

$$R(C_1) = L(C_1) - H(S) = 1.75 - 1.5 = 0.25 \text{ bit per simbolo,} \tag{1.22}$$

causata dal fatto che la parola associata a c (110) è inutilmente lunga.

1.5.3 Caso 2: Codice Ottimo ad Efficienza Massima

Riassegniamo la codifica per eliminare la ridondanza, abbreviando la codeword di c:

- $a \to 0 \quad (|c(a)| = 1 \text{ bit})$
- $b \to 10 \quad (|c(b)| = 2 \text{ bit})$
- $c \to 11 \quad (|c(c)| = 2 \text{ bit})$

Calcoliamo la lunghezza media del nuovo codice C_2:

$$\begin{aligned} L(C_2) &= p(a) \cdot 1 + p(b) \cdot 2 + p(c) \cdot 2 \\ &= \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{4} \cdot 2 \\ &= \frac{1}{2} + \frac{1}{2} + \frac{1}{2} = \frac{3}{2} = 1.5 \text{ bit per simbolo.} \end{aligned} \tag{1.23}$$

Nel codice C_2, la lunghezza media coincide esattamente con l'entropia della sorgente:

$$L(C_2) = H(S) = 1.5 \text{ bit per simbolo.} \tag{1.24}$$

Ogni ridondanza statistica è stata eliminata: il codice è a **efficienza teorica massima** (100%).

1.6 Rappresentazione ad Albero Binario e la Regola Aurea

1.6.1 Alberi di Codifica e Corrispondenza Geometrica

Qualsiasi codice privo di prefisso ammette una rappresentazione biunivoca mediante un **albero binario radicato**:

- Ciascun ramo sinistro corrisponde convenzionalmente all'etichetta binaria `0`;
- Ciascun ramo destro corrisponde convenzionalmente all'etichetta binaria `1`;
- I simboli dell'alfabeto A risiedono **esclusivamente nei nodi foglia**;
- La parola di codice $c(\sigma)$ associata a un simbolo σ è la sequenza ordinata di bit incontrati lungo il cammino radice-foglia.

Poiché nessun simbolo è collocato su un nodo interno, nessun cammino può proseguire oltre una foglia: la proprietà di prefisso è pertanto garantita dalla topologia stessa dell'albero.

1.6.2 Potatura dei Nodi Unari Ridondanti

La differenza strutturale tra il codice sub-ottimo C_1 e il codice ottimo C_2 si riflette immediatamente nella geometria dei rispettivi alberi binari (Figura 1.2).

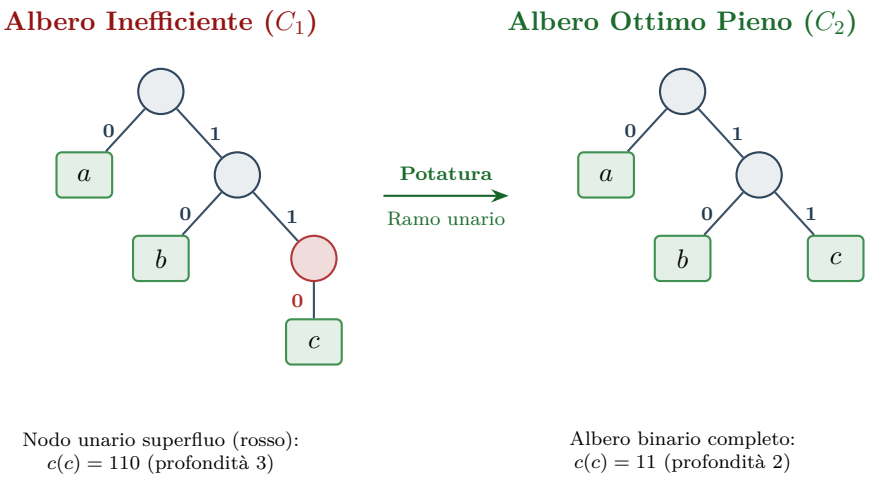


Figura 1.2: Confronto strutturale tra albero di codifica sub-ottimo (con nodo unario ridondante) e albero di codifica ottimo privo di prefisso. L'eliminazione del cammino superfluo verso la foglia c riduce la lunghezza della codeword da 3 a 2 bit, portando la lunghezza media del codice a coincidere con l'entropia della sorgente ($L(C) = H(S) = 1.5$ bit).

Nell'albero inefficiente di sinistra, il nodo interno contrassegnato in rosso possiede un unico figlio: esso allunga artificialmente la parola di c senza apportare alcuna distinzione di biforcazione. Potando il ramo unario e collegando direttamente c come figlio destro con etichetta `1`, l'albero diventa **binario pieno** (ogni nodo interno ha esattamente 2 figli), massimizzando l'efficienza.

1.6.3 La Regola Aurea della Compressione Dati (Golden Rule)

La condizione geometrica di ottimalità si traduce nel principio cardine dell'ingegneria della compressione:

La Regola Aurea (Golden Rule of Data Compression)

Probable symbols must be assigned shorter codewords.

Ai simboli dotati di probabilità più elevata devono essere tassativamente associate parole di codice più corte (foglie a profondità minore nell'albero).

Per apprezzare l'impatto della regola aurea, confrontiamo due distribuzioni di probabilità:

- Se le probabilità fossero $p(a) = \frac{1}{4}, p(b) = \frac{1}{2}, p(c) = \frac{1}{4}$, il simbolo più frequente sarebbe b: assegnare a b una parola da 2 bit e ad a una parola da 1 bit violerebbe la regola aurea, determinando una lunghezza media sub-ottima;
- Con la distribuzione effettiva $p(a) = \frac{1}{2}, p(b) = \frac{1}{4}, p(c) = \frac{1}{4}$, il simbolo dominante è a: ad esso viene riservata la parola di profondità minima (1 bit, cammino `0`), mentre a b e c spettano cammini di lunghezza 2 (`10` e `11`), minimizzando la media ponderata.

1.7 Architettura Fisica del File Compresso e Processo di Decodifica

La finalità pratica di qualsiasi sistema di compressione è produrre un flusso di byte persistente o trasmissibile:

"The output of Data Compression is a file." (1.25)

1.7.1 Formato Fisico: Preambolo e Corpo Compresso

Il file compresso generato presenta una rigida partizione binaria in due macro-sezioni contigue (Figura 1.3):

1. **Preambolo (Preamble / Header):** memorizza la tabella dei codici o la distribuzione delle frequenze necessaria per ricostruire l'albero di decodifica. Senza tale intestazione, un decodificatore privo di conoscenza a priori non potrebbe interpretare il flusso di bit;
2. **Corpo Compresso (Compressed Body):** contiene la sequenza effettiva di bit ottenuta concatenando le parole di codice corrispondenti ai simboli originali.

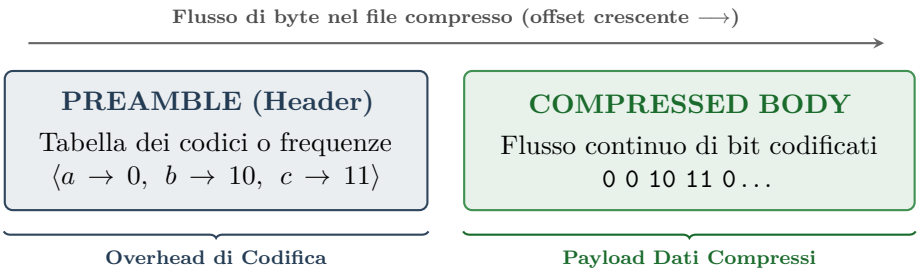


Figura 1.3: Architettura fisica del file compresso: il preambolo memorizza le corrispondenze simbolo-codice necessarie al decodificatore per ricostruire l'albero di decodifica, mentre il corpo compresso contiene la concatenazione delle parole di codice binarie dei dati originali.

1.7.2 Tracciamento Passo-Passo della Decodifica Istantanea

Consideriamo un file compresso strutturato come segue:

Preambolo (Preamble)	**Corpo Compresso (Compressed Body)**
$a \to 0, \quad b \to 10, \quad c \to 11$	`0 0 10 11 0`

Tabella 1.1: Struttura del messaggio compresso per la simulazione di decodifica.

Il decodificatore scansiona il corpo compresso bit per bit da sinistra verso destra, discendendo lungo l'albero di decodifica fino a toccare una foglia:

1. **1° bit (`0`):** cammino radice-foglia immediato. Simbolo decodificato: **a**. Ritorno alla radice;
2. **2° bit (`0`):** cammino radice-foglia immediato. Simbolo decodificato: **a**. Ritorno alla radice;
3. **3° e 4° bit (`10`):** il bit `1` scende nel figlio destro; il successivo `0` raggiunge la foglia b. Simbolo decodificato: **b**. Ritorno alla radice;
4. **5° e 6° bit (`11`):** il bit `1` scende a destra; il successivo `1` tocca la foglia c. Simbolo decodificato: **c**. Ritorno alla radice;
5. **7° bit (`0`):** tocca la foglia a. Simbolo decodificato: **a**.

La sequenza originale viene interamente e univocamente ricostruita:

$$\text{Testo Ricostruito: } \mathbf{a\ \ a\ \ b\ \ c\ \ a}. \tag{1.26}$$

1.8 Modelli per la Stima delle Probabilità

Poiché l'assegnazione ottima delle codeword dipende intimamente dalla distribuzione a priori $p(\sigma)$, qualsiasi compressore deve avvalersi di un **modello statistico** per stimare le probabilità dei simboli. La letteratura e la pratica ingegneristica distinguono tre paradigmi operativi fondamentali [2].

Modello Statistico	**Passaggi**	**Overhead Preambolo**	**Caratteristiche Operative e Punti di Forza**
Modello Statico (*Static Model*)	1 passaggio	Nullo (0 bit)	Probabilità prefissate a priori su standard storici del dominio (es. frequenze delle lettere nell'italiano). Encoder e decoder condividono la stessa tabella immutabile. Subottimale su file specifici.
Modello Semi-Statico (*Semi-Static Model*)	2 passaggi	Presente ($\mathcal{O}(\|A\| \log n)$ bit)	**Passo 1:** scansione completa del file per calcolare le frequenze esatte; **Passo 2:** generazione dell'albero ottimo e codifica. Obbliga a memorizzare l'albero/frequenze nel preambolo; non applicabile a data stream infiniti.
Modello Dinamico / Adattivo (*Dynamic Model*)	1 passaggio	Nullo	Probabilità aggiornate dinamicamente a runtime man mano che i simboli vengono letti. Encoder e decoder evolvono la struttura in perfetto sincronismo sullo storico condiviso. Si adatta alle variazioni locali del testo.

Tabella 1.2: Quadro comparativo dei modelli statistici per la stima delle probabilità nei sistemi di compressione.

Capitolo 2

Compressori Statistici, Codifica di Huffman e Forma Canonica

2.1 Introduzione ai Compressori Statistici e Modello Semi-Statico

Nei sistemi di compressione dati basati su modelli statistici, la progettazione del codice fa leva sulla frequenza empirica o sulla probabilità a priori dei simboli emessi dalla sorgente. All'interno di questa famiglia, l'approccio **semi-statico** (o a due passate) costituisce uno standard di riferimento fondamentale:

1. **Prima passata (Modellazione):** il codificatore scansiona l'intero flusso di ingresso per calcolare le frequenze assolute di emissione di ciascun simbolo e derivarne la distribuzione empirica di probabilità;
2. **Seconda passata (Codifica):** sulla base delle probabilità calcolate viene sintetizzato un codice ottimo privo di prefisso (*prefix-free code*), con cui il testo viene trasformato nel corpo del file compresso (sezione 2.5).

L'algoritmo progettato da David Huffman nel 1950 (e formalmente pubblicato nel 1952 [3]) rappresenta la soluzione esatta al problema di determinare, per una data distribuzione di probabilità su un alfabeto finito, un codice binario privo di prefisso avente la **minima lunghezza media attesa**.

2.1.1 Definizione Formale della Sorgente e Distribuzione Assegnata

Per analizzare le proprietà dell'algoritmo, consideriamo una sorgente discreta di informazione definita su un alfabeto finito composto da $|A| = 6$ simboli

distinti:

$$A = \{a, b, c, d, e, f\} \tag{2.1}$$

A ciascun simbolo $\sigma \in A$ è associata una probabilità di emissione nota a priori:

$$\begin{aligned} p(a) = 0.05, \quad p(b) = 0.10, \quad p(c) = 0.15, \\ p(d) = 0.30, \quad p(e) = 0.25, \quad p(f) = 0.15 \end{aligned} \tag{2.2}$$

La distribuzione soddisfa rigorosamente l'assioma di normalizzazione su spazi discreti:

$$\sum_{\sigma \in A} p(\sigma) = 0.05 + 0.10 + 0.15 + 0.30 + 0.25 + 0.15 = 1.00 \tag{2.3}$$

2.2 Costruzione dell'Albero di Huffman: Approccio Greedy

La sintesi dell'albero di codifica nell'algoritmo di Huffman segue una strategia **ingorda** (*greedy approach*) di tipo bottom-up. L'invariante costruttivo cardine può essere compendiato nella regola operativa:

"Merge first the smallest probabilities together." (2.4)

2.2.1 Derivazione Passo-Passo delle Fusioni

L'algoritmo mantiene una foresta di nodi pesati, in cui ciascun albero rappresenta un simbolo singolo o un raggruppamento composito. Ad ogni iterazione, vengono estratti i due nodi di peso minimo e fusi in un nuovo nodo padre la cui probabilità è pari alla somma dei pesi dei figli (la cui struttura finale converge nell'albero di Figura 2.1):

1. **Passo 1 (Fusione di a e b):** Tra i pesi disponibili nella foresta iniziale $\{a : 0.05,\ b : 0.10,\ c : 0.15,\ f : 0.15,\ e : 0.25,\ d : 0.30\}$, i due minimi assoluti sono a (0.05) e b (0.10). Viene generato il nodo intermedio ab di peso:
$$p(ab) = 0.05 + 0.10 = 0.15 \tag{2.5}$$
Foresta attiva: $\{ab : 0.15,\ c : 0.15,\ f : 0.15,\ e : 0.25,\ d : 0.30\}$.

2. **Passo 2 (Fusione di c e f):** I due nodi a peso minimo nella nuova configurazione sono c (0.15) e f (0.15). La loro fusione genera il nodo intermedio cf:
$$p(cf) = 0.15 + 0.15 = 0.30 \tag{2.6}$$
Foresta attiva: $\{ab : 0.15,\ e : 0.25,\ cf : 0.30,\ d : 0.30\}$.

3. **Passo 3 (Fusione di ab ed e):** I due elementi correnti a peso minimo sono il sottoalbero ab (0.15) e la foglia e (0.25). La loro unione dà origine al macro-nodo abe:

$$p(abe) = 0.15 + 0.25 = 0.40 \tag{2.7}$$

Foresta attiva: $\{cf : 0.30,\ d : 0.30,\ abe : 0.40\}$.

4. **Passo 4 (Fusione di cf e d):** I minimi correnti sono il nodo cf (0.30) e la foglia d (0.30). Viene generato il nodo composito cfd:

$$p(cfd) = 0.30 + 0.30 = 0.60 \tag{2.8}$$

Foresta attiva: $\{abe : 0.40,\ cfd : 0.60\}$.

5. **Passo 5 (Fusione Finale alla Radice):** I due sottoalberi residui vengono collegati alla radice dell'albero globale, completando la struttura:

$$p_{\text{root}} = 0.40 + 0.60 = 1.00 \tag{2.9}$$

2.2.2 Topologia dell'Albero ed Estrazione delle Parole di Codice

Fissata la convenzione topologica che associa il bit `0` ad ogni diramazione sinistra e il bit `1` ad ogni diramazione destra, l'albero risultante assume la configurazione illustrata in Figura 2.1.

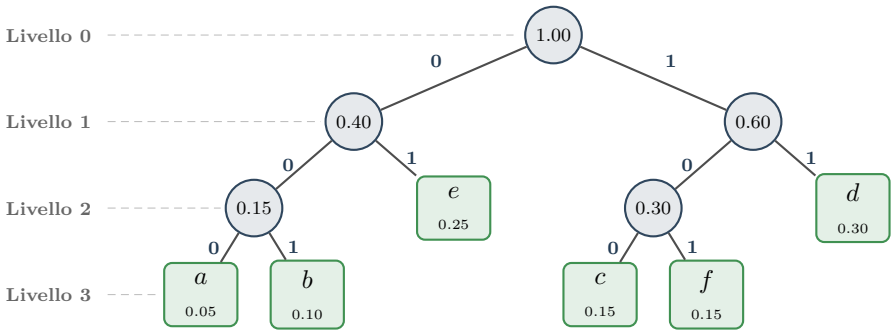


Figura 2.1: Albero binario di Huffman ottenuto dall'applicazione dell'algoritmo greedy sui 6 simboli della sorgente. I cerchi blu denotano i nodi interni con la probabilità cumulata, mentre i rettangoli verdi rappresentano le foglie associate ai singoli simboli con la rispettiva probabilità.

Seguendo il percorso orientato dalla radice verso ciascuna foglia, si deducono in modo univoco le relative parole di codice (*codewords*):

- **a** (Sinistra → Sinistra → Sinistra): `000` (lunghezza: 3 bit);
- **b** (Sinistra → Sinistra → Destra): `001` (lunghezza: 3 bit);
- **e** (Sinistra → Destra): `01` (lunghezza: 2 bit);
- **c** (Destra → Sinistra → Sinistra): `100` (lunghezza: 3 bit);
- **f** (Destra → Sinistra → Destra): `101` (lunghezza: 3 bit);
- **d** (Destra → Destra): `11` (lunghezza: 2 bit).

2.3 Analisi di Ottimalità e Limiti Teorici di Efficienza

2.3.1 Verifica Numerica della Lunghezza Media e Confronto Entropico

La lunghezza media attesa del codice di Huffman sintetizzato è calcolata come speranza matematica ponderata sulle probabilità dei simboli (Tabella 2.1):

$$L(C) = \sum_{\sigma \in A} p(\sigma) \cdot |c(\sigma)| \tag{2.10}$$

Tabella 2.1: Quadro sinottico delle parole di codice, lunghezze e contributi alla lunghezza media attesa.

Simbolo σ	**Probabilità** $p(\sigma)$	**Codeword** $c(\sigma)$	**Lunghezza** $\lvert c(\sigma)\rvert$	**Contributo** $p(\sigma)\lvert c(\sigma)\rvert$
a	0.05	`000`	3	$0.05 \times 3 = 0.15$ bit
b	0.10	`001`	3	$0.10 \times 3 = 0.30$ bit
c	0.15	`100`	3	$0.15 \times 3 = 0.45$ bit
d	0.30	`11`	2	$0.30 \times 2 = 0.60$ bit
e	0.25	`01`	2	$0.25 \times 2 = 0.50$ bit
f	0.15	`101`	3	$0.15 \times 3 = 0.45$ bit
Totale	**1.00**	—	—	$\mathbf{L(C) = 2.45}$ bit/simb

Calcolando l'entropia della sorgente $H(S)$ secondo la definizione di Shannon [1]:

$$\begin{aligned} H(S) &= -\sum_{\sigma \in A} p(\sigma) \log_2 p(\sigma) \\ &= -\Big(0.05 \log_2 0.05 + 0.10 \log_2 0.10 + 2 \times 0.15 \log_2 0.15 \\ &\quad + 0.25 \log_2 0.25 + 0.30 \log_2 0.30 \Big) \\ &\approx 0.2161 + 0.3322 + 0.8211 + 0.5000 + 0.5211 \approx \mathbf{2.3905} \text{ bit/simbolo} \end{aligned} \tag{2.11}$$

Si constata immediatamente che:

$$H(S) \approx 2.3905 \leq L(C) = 2.45 < H(S) + 1 \approx 3.3905 \tag{2.12}$$

La ridondanza del codice di Huffman rispetto all'ottimo ideale è contenuta in appena:

$$R = L(C) - H(S) \approx 2.45 - 2.3905 = 0.0595 \text{ bit/simbolo} \tag{2.13}$$

2.3.2 Il Limite Discreto: "We Cannot Fractionate the Bits"

Il teorema di Shannon sancisce che il limite inferiore teorico assoluto (*theoretic lower bound*) per la compressione priva di perdite è dato da $H(S)$. L'algoritmo di Huffman rappresenta il **miglior compressore reale a livello di singolo simbolo** (*real best compressor*):

- Per eguagliare esattamente l'entropia ($L(C) = H(S)$), ciascun simbolo dovrebbe essere codificato con un numero ideale di bit pari alla propria auto-informazione:
$$\ell^*(\sigma) = \log_2 \frac{1}{p(\sigma)} \tag{2.14}$$
- In qualsiasi compressore reale che opera mappando singoli simboli in parole binarie, la lunghezza del codice è vincolata all'insieme dei numeri interi positivi:
$$|c(\sigma)| \in \mathbb{N}^+ = \{1, 2, 3, \dots\} \tag{2.15}$$
Poiché non è fisicamente possibile frazionare i bit (*"Because we cannot fractionate the bits"*), $|c(\sigma)|$ costituisce un'approssimazione intera di $\log_2 \frac{1}{p(\sigma)}$, generando una discrepanza intrinseca ineliminabile.

2.3.3 Punti di Forza e Criticità Operative (Pros & Cons)

L'analisi ingegneristica dell'algoritmo di Huffman evidenzia un netto contrasto tra semplicità teorica e complessità computazionale di runtime:

- **Vantaggi (Pros):**
  1. *Semplicità algoritmica:* l'implementazione basata su code a priorità (*min-heap*) richiede tempo $\mathcal{O}(|A| \log |A|)$ (o $\mathcal{O}(|A|)$ se i simboli sono già ordinati per frequenza);
  2. *Ottimalità matematica dimostrata:* nessun altro codice privo di prefisso basato su simboli isolati può produrre una lunghezza media inferiore.
- **Svantaggi e Criticità (Cons):**
  1. *Decodifica intrinsecamente lenta:* nella decodifica standard basata su albero binario, ogni singolo bit in ingresso richiede il dereferenziamento di un puntatore di memoria (ramo `0` verso sinistra, ramo `1` verso destra);
  2. *Penalizzazioni da cache miss:* a causa dell'imprevedibilità statistica dei bit di input, i cammini attraverso i nodi dell'albero degradano la località dei riferimenti, innescando continui fallimenti della cache della CPU (*cache misses*) e stalli della pipeline di esecuzione (*branch mispredictions*).

2.4 Teorema sull'Efficienza di Huffman ed Effetto della Distribuzione

Teorema 2.4.1 (Teorema dell'Efficienza di Huffman)**.** *Sia S una sorgente discreta con alfabeto finito A ed entropia $H(S)$. La lunghezza media del codice di Huffman $L(C)$ è rigorosamente delimitata dalla doppia disuguaglianza:*

$$H(S) \leq L(C) < H(S) + 1 \tag{2.16}$$

Il teorema stabilisce che la lunghezza media del codice ottimo differisce dall'entropia teorica per **meno di 1 bit per simbolo**.

2.4.1 Dimensione Complessiva del File Compresso

Estendendo il risultato a una sequenza di m simboli emessi dalla sorgente ($m =$ # SIMBOLI DA COMPRIMERE), la dimensione complessiva del corpo del file compresso in bit soddisfa i limiti (Figura 2.2):

$$m \cdot H(S) \leq \text{Dimensione Compressa (bit)} < m \cdot H(S) + m \tag{2.17}$$

Il termine $+m$ rappresenta il **sovraccosto additivo massimo** (*extra cost w.r.t. entropy*) generato dall'arrotondamento all'intero della lunghezza delle parole di codice.

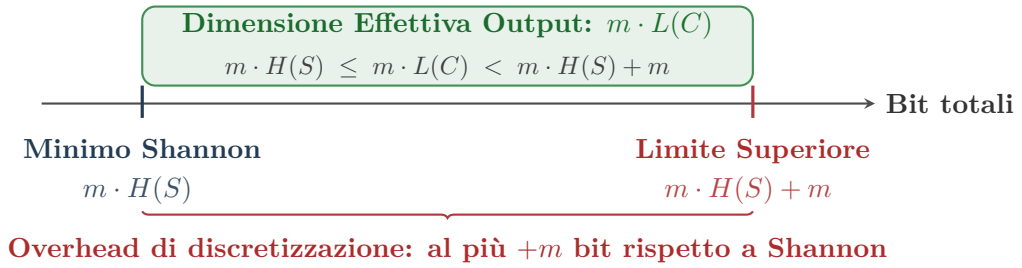


Figura 2.2: Delimitazione della dimensione dell'output di Huffman per m simboli: il corpo compresso è rigorosamente compreso tra il minimo teorico shannoniano $m \cdot H(S)$ e il tetto massimo $m \cdot H(S) + m$ bit.

2.4.2 Comportamento Asintotico al Variare della Distribuzione

L'impatto percentuale dell'overhead di 1 bit per simbolo diverge radicalmente a seconda della forma della distribuzione di probabilità:

1. **Sorgente Fortemente Sbilanciata ($H(S) \to 0$, Skewed Source):**
   - Si consideri una sorgente in cui un singolo simbolo domina quasi per intero il flusso (ad es. $p(\sigma_1) = 0.999$, $p(\sigma_2) = 0.001$), per cui l'entropia reale è prossima a zero: $H(S) \approx 0.01$ bit/simbolo;
   - Poiché nessun codice privo di prefisso può assegnare a un simbolo una parola di lunghezza inferiore a 1 bit ($|c(\sigma)| \geq 1$), Huffman è costretto ad allocare almeno 1 bit per ciascun simbolo;
   - In tale regime, **Huffman offre prestazioni scadenti** (*"Huffman is bad / loses a lot"*): spendere 1 bit a fronte di un limite teorico di 0.01 bit comporta un incremento di dimensione pari a 100 volte

il limite di Shannon (+10000%). In questi scenari si rivelano indispensabili compressori a blocchi (Run-Length Encoding) o compressori aritmetici/ANS.

2. **Sorgente Uniforme ($H(S) \to \log_2 |A|$, Uniform Source):**

   - Si consideri una sorgente con distribuzione quasi uniforme su un alfabeto ampio (ad es. byte, $|A| = 256$), in cui $H(S) \approx \log_2 256 = 8$ bit/simbolo;
   - In questa situazione, la sorgente è intrinsecamente poco comprimibile, e le parole di codice di Huffman hanno lunghezze intorno a 8 bit;
   - L'overhead additivo di frazione di bit incide in misura del tutto trascurabile (*"+1 bit is nothing"*): un costo aggiuntivo di al più 1 bit su una parola da 8 bit corrisponde a una deviazione percentuale minima. In tale scenario **Huffman è pienamente soddisfacente**.

2.5 Architettura del File Semi-Statico e Limiti degli Alberi Standard

Nelle applicazioni reali, il flusso compresso prodotto da un modello statistico semi-statico deve essere memorizzato o trasmesso in un file autosufficiente:

$$\text{File Compresso} = \Big[\, \text{PREAMBLE} \;\Big|\; \text{COMPRESSED BODY} \,\Big] \tag{2.18}$$

Quando l'alfabeto di riferimento è orientato ai byte (standard de facto nei sistemi informatici reali):

$$A = \text{BYTE} \implies |A| = 256 = 2^8 \text{ simboli} \tag{2.19}$$

Memorizzare direttamente l'albero di Huffman standard con la sua topologia e i puntatori all'interno del preambolo comporta tre gravi criticità:

1. **Overhead di memoria elevato:** serializzare la struttura di puntatori di un albero con centinaia di nodi consuma prezioso spazio nel preambolo, vanificando il guadagno di compressione sui file di piccole o medie dimensioni;

2. **Incompatibilità tra architetture:** la serializzazione dei puntatori richiede convenzioni su indirizzamento ed endianness;

3. **Decodifica inefficiente:** il decodificatore deve ricostruire l'albero dinamico in RAM e navigarlo bit per bit con dereferenziazione di puntatori, saturando la banda della cache del processore.

2.6 Proprietà Strutturali e Combinatorie degli Alberi di Huffman

L'analisi geometrica e combinatoria degli alberi binari di Huffman rivela tre proprietà fondamentali.

Proprietà 2.6.1 (Numero di Nodi Interni)**.** *In qualsiasi albero binario completo (strictly binary tree) con $|A|$ foglie, il numero complessivo di nodi interni è invariabilmente pari a:*

$$\#NODI\ INTERNI = |A| - 1 \tag{2.20}$$

Dimostrazione. Ogni iterazione dell'algoritmo di Huffman estrae esattamente 2 nodi e inserisce 1 nuovo nodo padre intermedio, riducendo la cardinalità della foresta di 1. Partendo da $|A|$ radici e arrestandosi a 1 radice finale, si compiono esattamente $|A| - 1$ fusioni, ciascuna delle quali genera un distinto nodo interno. □

Proprietà 2.6.2 (Molteplicità di Codici Ottimi Equivalenti)**.** *Dato un albero di Huffman con $|A|$ foglie, esistono esattamente:*

$$2^{|A|-1}\ \textit{codici ottimi distinti} \tag{2.21}$$

aventi la medesima lunghezza media attesa $L(C)$ e le stesse lunghezze delle parole di codice.

Dimostrazione. Ciascuno dei $|A| - 1$ nodi interni possiede due rami uscenti a cui vengono assegnate le etichette binarie `0` e `1`. Invertendo arbitrariamente l'assegnazione delle etichette (ramo sinistro = 1, ramo destro = 0) su un qualsiasi nodo interno, le profondità di tutte le foglie rimangono inalterate e la proprietà di prefisso è rigorosamente conservata. Poiché vi sono $|A| - 1$ scelte binarie indipendenti, lo spazio dei codici isomorfi ha cardinalità $2^{|A|-1}$. □

Proprietà 2.6.3 (Non Esaustività della Procedura Standard)**.** *Esistono configurazioni di codici ottimi a lunghezza variabile che soddisfano il minimo teorico di lunghezza media ma che non possono essere derivate direttamente dall'esecuzione deterministica dell'algoritmo greedy standard di Huffman:*

$$\exists\, C^*\ \textit{ottimo} \quad \textit{tale che} \quad C^* \notin \textit{HuffmanStandard}(A, p) \tag{2.22}$$

2.7 Codifica Canonica di Huffman (Canonical Huffman)

La **codifica canonica di Huffman** (sviluppata formalmente da Schwartz e Kallick e perfezionata da Hirschberg e Lelewer [4, 5]) risolve in modo definitivo i limiti prestazionali e di occupazione di memoria degli alberi binari standard:

- Sfrutta il fatto che l'ottimalità del codice dipende **esclusivamente dall'insieme delle lunghezze** delle parole di codice $\{|c(\sigma)|\}$, e non dalla specifica conformazione geometrica dell'albero;
- Riorganizza le parole di codice secondo una sequenza numerica strettamente ordinata e compatta;
- Elimina completamente la necessità di memorizzare o navigare alberi a puntatori, consentendo la decodifica istantanea basata su comparazioni aritmetiche tra registri interi.

2.7.1 Definizione delle Strutture Dati Fondamentali

Sia L la massima lunghezza di parola di codice presente nell'albero ($L = \max_\sigma |c(\sigma)|$). Il formalismo canonico definisce tre strutture dati di supporto:

Definizione 2.7.1 (Vettore NUM)**.** Il vettore $\mathrm{NUM}[1 \dots L]$ è un array di interi in cui ogni elemento $\mathrm{NUM}[\ell]$ indica il numero totale di simboli dell'alfabeto associati a parole di codice di lunghezza esattamente pari a ℓ:

$$\mathrm{NUM}[\ell] = \Big|\{\sigma \in A : |c(\sigma)| = \ell\}\Big|, \quad \forall \ell \in \{1, \dots, L\} \tag{2.23}$$

Definizione 2.7.2 (Tabella SYMB)**.** La struttura $\mathrm{SYMB}[1 \dots L]$ memorizza, per ciascuna lunghezza ℓ, l'elenco ordinato (lessicograficamente) dei simboli aventi parola di codice di lunghezza ℓ:

$$\mathrm{SYMB}[\ell] = \langle \sigma_{i_1}, \sigma_{i_2}, \dots, \sigma_{i_{\mathrm{NUM}[\ell]}} \rangle \tag{2.24}$$

Definizione 2.7.3 (Vettore FC (First Code))**.** Il vettore $FC[1 \dots L]$ memorizza il valore intero associato alla **prima parola di codice** (codice lessicograficamente minimo) di lunghezza ℓ.

2.8 Esempio Completo di Costruzione del Codice Canonico

Si consideri l'esempio didattico analizzato nel corso, basato su un alfabeto di $|A| = 7$ simboli:

$$A = \{a, b, c, d, e, f, g\} \tag{2.25}$$

con un albero di Huffman di profondità massima $L = 4$. Il numero di nodi interni è #NODI INTERNI $= 7 - 1 = 6$.

2.8.1 Calcolo dei Vettori NUM e SYMB

Analizzando la profondità a cui sono collocate le foglie nell'albero di partenza:

- Livello $\ell = 1$: 1 foglia $(g) \implies$ NUM[1] $= 1$, SYMB[1] $= [g]$;
- Livello $\ell = 2$: nessuna foglia $\implies$ NUM[2] $= 0$, SYMB[2] $= \emptyset$;
- Livello $\ell = 3$: 2 foglie $(c, d) \implies$ NUM[3] $= 2$, SYMB[3] $= [c, d]$;
- Livello $\ell = 4$: 4 foglie $(a, b, e, f) \implies$ NUM[4] $= 4$, SYMB[4] $= [a, b, e, f]$.

Il vettore complessivo delle occorrenze di lunghezza è:

$$\text{NUM}[1 \dots 4] = [1,\ 0,\ 2,\ 4] \tag{2.26}$$

2.8.2 Calcolo Ricorsivo di $FC[\ell]$ (Formula Bottom-Up)

Nella formalizzazione adottata nelle lezioni di Algorithm Engineering, il vettore dei primi codici $FC[\ell]$ viene calcolato risalendo dall'estremità inferiore dell'albero (livello massimo $L = 4$) verso la radice ($\ell = 1$), applicando la formula di ricorrenza:

$$\begin{cases} FC[L] = 0 \\ FC[\ell - 1] = \left\lfloor \dfrac{FC[\ell] + \text{NUM}[\ell]}{2} \right\rfloor, \quad \text{per } \ell = L, L-1, \dots, 2 \end{cases} \tag{2.27}$$

Applicando la formula passo dopo passo:

1. **Inizializzazione ($\ell = 4$):**

$$FC[4] = 0 \qquad (0000_2) \tag{2.28}$$

2. **Livello $\ell = 3$:**

$$FC[3] = \left\lfloor \frac{FC[4] + \text{NUM}[4]}{2} \right\rfloor = \left\lfloor \frac{0+4}{2} \right\rfloor = \mathbf{2} \qquad (010_2) \qquad (2.29)$$

3. **Livello $\ell = 2$:**

$$FC[2] = \left\lfloor \frac{FC[3] + \text{NUM}[3]}{2} \right\rfloor = \left\lfloor \frac{2+2}{2} \right\rfloor = \mathbf{2} \qquad (10_2) \qquad (2.30)$$

4. **Livello $\ell = 1$:**

$$FC[1] = \left\lfloor \frac{FC[2] + \text{NUM}[2]}{2} \right\rfloor = \left\lfloor \frac{2+0}{2} \right\rfloor = \mathbf{1} \qquad (1_2) \qquad (2.31)$$

Il quadro riassuntivo dei parametri canonici per ciascun livello è sintetizzato in Tabella 2.2.

Tabella 2.2: Quadro riepilogativo delle strutture canoniche: livello ℓ, conteggio NUM, primo codice FC e simboli associati.

Livello ℓ	**NUM**$[\ell]$	$FC[\ell]$ **(Decimale)**	$FC[\ell]$ **(Binario)**	**Simboli SYMB**$[\ell]$
4	4	0	0000_2	$[a, b, e, f]$
3	2	2	010_2	$[c, d]$
2	0	2	10_2	$\emptyset$ (nessun simbolo)
1	1	1	1_2	$[g]$

2.8.3 Analisi Critica: I "Prefissi Fittizi" e la Correttezza Formale

Osservando il valore calcolato per il livello 2 ($FC[2] = 2$, ovvero 10_2), emerge un'interessante proprietà discussa dal docente:

Annotazione del Docente sui Prefissi Fittizi

"Apparently we create a bad prefix, but it will not play any role, so it is correct."

Spiegazione formale della correttezza: A prima vista, il valore $FC[2] = 2$ (10_2) sembrerebbe violare la proprietà di prefisso rispetto ad altri cammini. Tuttavia, poiché al livello $\ell = 2$ non esiste alcun simbolo reale ($\text{NUM}[2] = 0$ e $\text{SYMB}[2] = \emptyset$), il decodificatore non emetterà mai alcun simbolo a questa lunghezza. Il valore $FC[2] = 2$ funge puramente da **soglia aritmetica di transito** che permette alla formula di ricorrenza di calcolare correttamente la soglia per il livello superiore ($FC[1] = 1$). Non essendoci simboli attivi a lunghezza 2, non può generarsi alcuna ambiguità di decodifica.

2.9 Decodifica Senza Albero (Tree-less Decoding)

L'innovazione applicativa fondamentale della codifica canonica risiede nella possibilità di eseguire la decodifica del flusso binario **senza ricostruire alcun albero in memoria**, utilizzando unicamente registri interi della CPU e lookup su array contigui.

2.9.1 Schema Funzionale della Decodifica

La decodifica di un singolo simbolo dal corpo compresso opera iterativamente su una variabile accumulatore intera V, inizializzata a 0, leggendo il flusso binario un bit alla volta:

1. Si inizializzano l'accumulatore $V \leftarrow 0$ e la lunghezza corrente $\ell \leftarrow 1$;
2. Per ciascun bit letto, si aggiorna l'accumulatore tramite shift a sinistra: $V \leftarrow (V \ll 1) + \text{bit} \equiv 2V + \text{bit}$;
3. Si confronta V con il primo codice valido del livello corrente, $FC[\ell]$:
   - Se $V < FC[\ell]$, la sequenza esaminata non costituisce ancora una parola di codice terminata: si incrementa la lunghezza $\ell \leftarrow \ell + 1$ e si ripete il procedimento sul bit successivo;
   - Se $V \geq FC[\ell]$, la parola di codice è stata identificata: l'indice del simbolo all'interno del livello è dato dallo scostamento $\text{offset} = V - FC[\ell]$, e l'algoritmo restituisce direttamente il simbolo $\text{SYMB}[\ell][\text{offset}]$.

2.9.2 Diagramma di Flusso della Decodifica

Il flusso operativo della decodifica a registri è formalizzato nel diagramma in Figura 2.3.

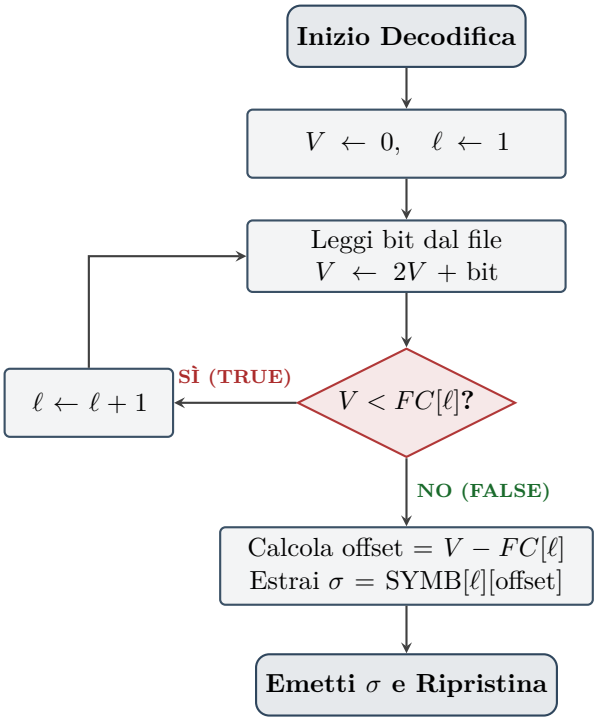


Figura 2.3: Diagramma di flusso della decodifica canonica senza albero: ad ogni bit letto, l'accumulatore intero V viene aggiornato e confrontato con la soglia di arresto $FC[\ell]$.

2.9.3 Tracciamento Passo-Passo dell'Esempio: Flusso `0 0 1 0`

Si consideri ora il flusso binario in arrivo descritto negli appunti:

$$\text{Flusso di bit in ingresso:} \quad \texttt{0 0 1 0} \tag{2.32}$$

L'algoritmo di decodifica canonica elabora la sequenza bit per bit (Tabella 2.3):

Tabella 2.3: Tracciamento analitico dell'esecuzione dell'algoritmo Tree-less sulla sequenza `0 0 1 0`.

Iterazione	**Bit Letto**	**Accumulatore** V	**Soglia** $FC[\ell]$	**Test:** $V < FC[\ell]$	**Esito**	**Azione Intrapresa**
$\ell = 1$	0	$2(0) + 0 = \mathbf{0}$	$FC[1] = 1$	$0 < 1$	**TRUE**	Parola non completa $\implies \ell \leftarrow 2$
$\ell = 2$	0	$2(0) + 0 = \mathbf{0}$	$FC[2] = 2$	$0 < 2$	**TRUE**	Parola non completa $\implies \ell \leftarrow 3$
$\ell = 3$	1	$2(0) + 1 = \mathbf{1}$	$FC[3] = 2$	$1 < 2$	**TRUE**	Parola non completa $\implies \ell \leftarrow 4$
$\ell = 4$	0	$2(1) + 0 = \mathbf{2}$	$FC[4] = 0$	$2 < 0$	**FALSE**	**Parola terminata! Arresto a** $\ell = 4$

All'arresto al livello $\ell = 4$, l'algoritmo calcola l'indice di selezione del simbolo tramite semplice sottrazione:

$$\text{offset} = V - FC[4] = 2 - 0 = \mathbf{2} \tag{2.33}$$

Accedendo al vettore SYMB[4] $= [a, b, e, f]$ con offset 2 (assumendo indicizzazione da 0: offset $0 \to a,\ 1 \to b,\ 2 \to e,\ 3 \to f$):

$$\text{Simbolo Emesso} = \text{SYMB}[4][2] \implies \mathbf{e} \tag{2.34}$$

Il simbolo corrispondente alla sequenza di 4 bit `0010` è dunque estratto con sole 4 moltiplicazioni per 2, 4 somme e 4 comparazioni intere, senza alcuna consultazione di tabelle di puntatori.

2.10 Valutazione Architetturale e Prospettive Avanzate

L'adozione della codifica canonica di Huffman introduce un duplice vantaggio sistemistico fondamentale:

1. **Compressione estrema del preambolo:** per consentire la decodifica non serve più serializzare l'albero di Huffman. È sufficiente memorizzare per ogni simbolo dell'alfabeto la sua **lunghezza di codice in bit** $|c(\sigma)|$. Dal vettore delle lunghezze, il decodificatore ricostruisce in modo deterministico e immediato i vettori NUM, SYMB e FC tramite la ricorrenza vista sopra. Con un alfabeto da 256 byte e lunghezze limitate a 16 bit, il preambolo occupa meno di 128 byte;

2. **Massimizzazione del Throughput di Decodifica:** l'eliminazione dei puntatori trasforma l'algoritmo in un ciclo aritmetico perfettamente vettorizzabile dalla CPU, azzerando le fallanze di cache.

2.10.1 Problemi Aperti: Allineamento e Perdita dei Bit Significativi

Nelle architetture concrete a 32 e 64 bit, la gestione dei registri di scorrimento presenta sfide implementative non banali:

- **Perdita del bit più significativo (*losing significant bit*):** durante le continue operazioni di shift $V \leftarrow (V \ll 1) + \text{bit}$, superata la larghezza di parola del registro macchina occorre gestire il trabocco (*overflow*) o segmentare lo scorrimento con buffer circolari;

- **Transizione verso compressori moderni (ANS):** sebbene Huffman rimanga insostituibile per semplicità e prestazioni su sorgenti uniformi, per superare il limite teorico dell'overhead di 1 bit su sorgenti fortemente sbilanciate la ricerca moderna ha introdotto i sistemi numerici asimmetrici (*Asymmetric Numeral Systems*, ANS [6]), che uniscono la velocità a registri di Huffman con la precisione frazionaria della codifica aritmetica.

Capitolo 3

Codifica Aritmetica (*Arithmetic Coding*)

3.1 Principi Fondamentali e Procedura Converter

La **codifica aritmetica** (*Arithmetic Coding*), introdotta formalmente da Rissanen [7] e perfezionata da Witten, Neal e Cleary [8], costituisce una tecnica fondamentale di compressione statistica priva di perdite. Essa supera radicalmente il vincolo architetturale dei codici a prefisso classici (come l'algoritmo di Huffman [3]): l'obbligo di associare a ciascun singolo simbolo un numero intero di bit.

Nel modello shannoniano [1], la quantità ideale di informazione generata da un simbolo σ con probabilità a priori $p(\sigma)$ è espressa dalla propria auto-informazione:

$$\ell^*(\sigma) = \log_2 \frac{1}{p(\sigma)} \text{ bit} \tag{3.1}$$

Tale valore è in generale un numero reale frazionario. Poiché i compressori a livello di singolo simbolo non possono emettere frazioni di bit, essi incorrono in un arrotondamento discreto all'intero superiore per ciascun carattere emesso. Al contrario, la codifica aritmetica elabora l'intera sequenza di testo T in modo globale, mappando l'intero messaggio in un **unico sotto-intervallo semiaperto** del dominio reale unitario:

$$[l, l + s) \subset [0, 1) \tag{3.2}$$

dove l rappresenta l'estremo inferiore (*base* o *lower bound*) ed s la larghezza (*size* o ampiezza) dell'intervallo. Il corpo compresso finale consiste nella

rappresentazione binaria frazionaria di un singolo punto numerico interno a tale intervallo.

3.1.1 Definizione Formale della Procedura CONVERTER

La generazione fisica dei bit di output viene delegata a una procedura formale denominata `CONVERTER`:

$$\text{CONVERTER}(x, k) \longrightarrow \langle b_1, b_2, \ldots, b_k \rangle \tag{3.3}$$

La procedura riceve in ingresso due argomenti:

1. Un valore reale $x \in [0, 1)$, selezionato internamente all'intervallo finale $[l, l + s)$ generato dal processo di compressione;
2. Un intero positivo $k \in \mathbb{N}^+$, che definisce il numero di cifre binarie di precisione richieste.

L'output coincide con le prime k cifre dell'espansione binaria frazionaria di x:

$$x \approx \sum_{j=1}^{k} b_j \cdot 2^{-j} = 0.b_1 b_2 \ldots b_k, \quad \text{con } b_j \in \{0, 1\} \tag{3.4}$$

3.1.2 Algoritmo per Moltiplicazioni Successive per Due

Il calcolo effettivo delle cifre binarie frazionarie $b_1, \ldots, b_k$ segue l'algoritmo classico di moltiplicazione ricorsiva per la base 2:

1. Si pone $x_1 = x$;
2. Per ciascun indice di iterazione $j = 1, 2, \ldots, k$:
   - Si raddoppia il valore corrente:
   $$y_j = 2 \cdot x_j \tag{3.5}$$
   - Il bit b_j è dato dalla parte intera inferiore di y_j:
   $$b_j = \lfloor y_j \rfloor \in \{0, 1\} \tag{3.6}$$
   - Si aggiorna il residuo frazionario per il passo successivo:
   $$x_{j+1} = y_j - b_j = \text{frac}(y_j) \tag{3.7}$$

3.2 Lemma sull'Approssimazione Binaria e Troncamento

L'emissione di una sequenza finita di d bit corrisponde matematicamente a un'operazione di troncamento della parte frazionaria. Il seguente risultato analitico quantifica rigorosamente l'errore commesso.

Lemma 3.2.1 (Lemma del Troncamento Binario)**.** *Sia $x \in [0, 1)$ un numero reale avente espansione binaria frazionaria infinita:*

$$x = 0.b_1 b_2 \dots b_d b_{d+1} b_{d+2} \cdots = \sum_{j=1}^{\infty} b_j \cdot 2^{-j} \tag{3.8}$$

Sia $\hat{x}$ la sua approssimazione troncata alle prime d cifre binarie:

$$\hat{x} = 0.b_1 b_2 \dots b_d 000 \cdots = \sum_{j=1}^{d} b_j \cdot 2^{-j} \tag{3.9}$$

Allora l'errore assoluto di troncamento è superiormente delimitato da:

$$x - \hat{x} \leq 2^{-d} \tag{3.10}$$

Dimostrazione. La differenza algebrica tra il valore esatto x e il valore troncato $\hat{x}$ è pari alla somma infinita dei pesi posizionali dei bit azzerati a partire dall'indice $d + 1$:

$$x - \hat{x} = \sum_{i=1}^{\infty} b_{d+i} \cdot 2^{-(d+i)} \tag{3.11}$$

Poiché ogni cifra binaria b_{d+i} appartiene all'insieme $\{0, 1\}$, si ha $b_{d+i} \leq 1$ per ogni $i \geq 1$:

$$x - \hat{x} \leq \sum_{i=1}^{\infty} 1 \cdot 2^{-(d+i)} \tag{3.12}$$

Raccogliendo a fattor comune il termine costante 2^{-d}, indipendente dall'indice di sommatoria:

$$x - \hat{x} \leq 2^{-d} \sum_{i=1}^{\infty} 2^{-i} \tag{3.13}$$

La sommatoria residua coincide con la serie geometrica di ragione $q = 1/2$, la cui somma converge a 1:

$$\sum_{i=1}^{\infty} 2^{-i} = \sum_{i=1}^{\infty} \left(\frac{1}{2}\right)^i = \frac{\frac{1}{2}}{1 - \frac{1}{2}} = \frac{\frac{1}{2}}{\frac{1}{2}} = 1 \tag{3.14}$$

Sostituendo il valore calcolato nella disuguaglianza si ottiene la tesi:

$$x - \hat{x} \leq 2^{-d} \cdot 1 = 2^{-d} \tag{3.15}$$

□

3.3 Processo di Compressione per Suddivisione Progressiva

La codifica comprime un messaggio trasformando sequenzialmente ciascun simbolo in una contrazione geometrica dell'intervallo reale.

3.3.1 Modello di Sorgente e Dati Operativi

Si consideri a titolo didattico una sorgente discreta definita sull'alfabeto ternario $A = \{a, b, c\}$, associata alla seguente distribuzione di probabilità:

- $p(a) = \dfrac{1}{2} = 0.50$
- $p(b) = \dfrac{1}{4} = 0.25$
- $p(c) = \dfrac{1}{4} = 0.25$

La somma delle probabilità soddisfa la condizione di normalizzazione:

$$\sum_{\sigma \in A} p(\sigma) = \frac{1}{2} + \frac{1}{4} + \frac{1}{4} = 1 \tag{3.16}$$

L'obiettivo è comprimere il testo:

$$T = a\,b\,c \quad (\text{lunghezza } m = 3) \tag{3.17}$$

3.3.2 Regole Generali di Aggiornamento Ricorsivo

All'istante iniziale (passo $k = 0$), prima di analizzare qualsiasi carattere, l'intervallo operativo coincide con l'intero segmento unitario:

$$l_0 = 0, \quad S_0 = 1 \implies [l_0, l_0 + S_0) = [0, 1) \tag{3.18}$$

Ad ogni passo $k \geq 1$, leggendo il carattere $T[k] = \sigma \in A$, l'intervallo attivo $[l_{k-1}, l_{k-1} + S_{k-1})$ viene partizionato in sotto-segmenti contigui e disgiunti di dimensione proporzionale alle probabilità dei simboli:

1. **Aggiornamento dell'ampiezza (*size*):**

$$S_k = S_{k-1} \cdot p(\sigma) \tag{3.19}$$

2. **Aggiornamento della base (*lower bound*):**

$$l_k = l_{k-1} + S_{k-1} \cdot \sum_{\tau < \sigma} p(\tau) \tag{3.20}$$

dove la sommatoria cumula le probabilità di tutti i simboli τ che precedono σ nell'ordine alfabetico fissato.

3.3.3 Tracciamento Passo-Passo per la Stringa $T = abc$

L'esecuzione dettagliata della procedura di codifica per la sequenza $T = a\,b\,c$ procede attraverso tre stadi successivi (Figura 3.1):

1. **Passo 1: Elaborazione del simbolo $T[1] = a$**
   L'intervallo di partenza $[0, 1)$ viene ripartito secondo le probabilità dell'alfabeto:

   - Sotto-intervallo per a: si estende da 0 fino a $p(a) = \frac{1}{2}$;
   - Sotto-intervallo per b: si estende da $\frac{1}{2}$ fino a $\frac{1}{2} + p(b) = \frac{3}{4}$;
   - Sotto-intervallo per c: si estende da $\frac{3}{4}$ fino a $\frac{3}{4} + p(c) = 1$.

   Poiché il simbolo letto è a, il codificatore seleziona il primo segmento:

   $$l_1 = 0, \quad S_1 = 1 \cdot p(a) = \frac{1}{2} \implies [l_1, l_1 + S_1) = \left[0, \frac{1}{2}\right) \tag{3.21}$$

2. **Passo 2: Elaborazione del simbolo $T[2] = b$**
   L'intervallo corrente $[0, 1/2)$ (ampiezza $S_1 = 1/2$) viene suddiviso internamente:

   - Segmento per a: ampiezza $p(a) \cdot S_1 = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$, copre $\left[0, \frac{1}{4}\right)$;
   - Segmento per b: ampiezza $p(b) \cdot S_1 = \frac{1}{4} \cdot \frac{1}{2} = \frac{1}{8}$, copre $\left[\frac{1}{4}, \frac{1}{4} + \frac{1}{8}\right) = \left[\frac{1}{4}, \frac{3}{8}\right)$;
   - Segmento per c: ampiezza $p(c) \cdot S_1 = \frac{1}{4} \cdot \frac{1}{2} = \frac{1}{8}$, copre $\left[\frac{3}{8}, \frac{1}{2}\right)$.

   Leggendo b, il codificatore seleziona il sotto-intervallo centrale:

   $$l_2 = \frac{1}{4}, \quad S_2 = S_1 \cdot p(b) = \frac{1}{2} \cdot \frac{1}{4} = \frac{1}{8} \implies [l_2, l_2 + S_2) = \left[\frac{1}{4}, \frac{3}{8}\right) \tag{3.22}$$

3. **Passo 3: Elaborazione del simbolo $T[3] = c$**
   Per analizzare chiaramente la suddivisione di $[1/4, 3/8)$, si riducono le frazioni al denominatore comune 32: la base è $l_2 = 1/4 = 8/32$ e l'ampiezza è $S_2 = 1/8 = 4/32$, per cui l'intervallo è $[8/32, 12/32)$:

   - Segmento per a: ampiezza $p(a) \cdot S_2 = \frac{1}{2} \cdot \frac{1}{8} = \frac{2}{32}$, copre $\left[\frac{8}{32}, \frac{10}{32}\right)$;

- Segmento per b: ampiezza $p(b) \cdot S_2 = \dfrac{1}{4} \cdot \dfrac{1}{8} = \dfrac{1}{32}$, copre $\left[\dfrac{10}{32}, \dfrac{11}{32}\right)$;
- Segmento per c: ampiezza $p(c) \cdot S_2 = \dfrac{1}{4} \cdot \dfrac{1}{8} = \dfrac{1}{32}$, copre $\left[\dfrac{11}{32}, \dfrac{12}{32}\right)$.

Il terzo simbolo è c, inducendo la selezione del segmento superiore:

$$l_3 = \frac{11}{32}, \quad S_3 = S_2 \cdot p(c) = \frac{1}{8} \cdot \frac{1}{4} = \frac{1}{32} \implies [l_3, l_3 + S_3) = \left[\frac{11}{32}, \frac{12}{32}\right) \tag{3.23}$$

Al termine dell'elaborazione della stringa $T = abc$, l'intervallo finale risultante è:

$$\text{COMPRESS}(T) = \left[\frac{11}{32}, \frac{12}{32}\right) \quad \text{con } l = \frac{11}{32} \text{ e } s = \frac{1}{32} \tag{3.24}$$

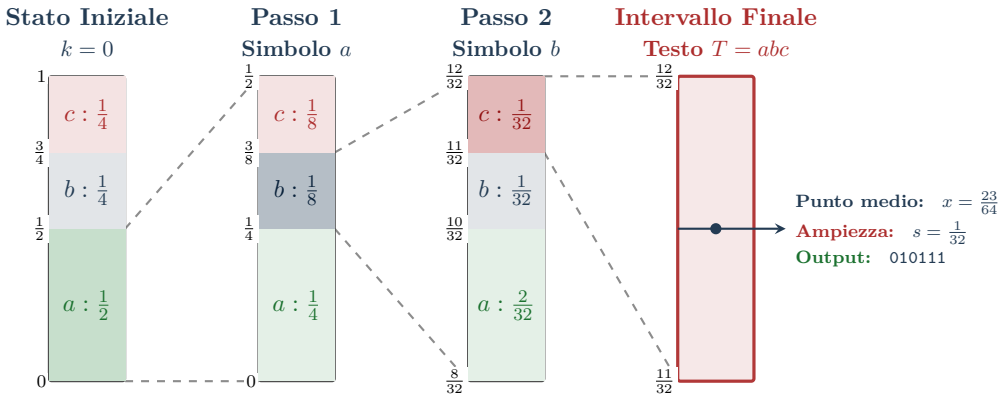


Figura 3.1: Suddivisione progressiva dell'intervallo reale unitario durante la codifica del testo $T = abc$: a ciascun passo il sotto-intervallo selezionato viene dilatato concettualmente e suddiviso proporzionalmente alle probabilità dei simboli.

3.4 Generazione dell'Output Compresso e Punto Medio

Ottenuto l'intervallo compresso $[l, l + s)$, il codificatore deve identificare un valore numerico che lo rappresenti univocamente all'interno del flusso di bit finale.

3.4.1 Formula di Emissione dell'Output

La regola generale stabilisce di codificare il **punto medio** dell'intervallo, impiegando un numero finito d di bit calibrato in funzione dell'ampiezza s:

$$\text{OUTPUT: CONVERTER}\left(l + \frac{s}{2},\ \left\lceil \log_2 \frac{2}{s} \right\rceil\right) \tag{3.25}$$

L'espressione individua due quantità determinanti:

1. **La coordinata del punto centrale dell'intervallo:**

$$x = l + \frac{s}{2} \tag{3.26}$$

2. **Il numero di bit di precisione richiesti:**

$$d = \left\lceil \log_2 \frac{2}{s} \right\rceil \tag{3.27}$$

3.4.2 Calcolo Analitico per il Caso di Studio $T = abc$

Applicando la regola ai parametri calcolati per $T = abc$ ($l = 11/32$, $s = 1/32$):

- **Determinazione del punto medio x:**

$$x = l + \frac{s}{2} = \frac{11}{32} + \frac{\frac{1}{32}}{2} = \frac{11}{32} + \frac{1}{64} = \frac{22}{64} + \frac{1}{64} = \mathbf{\frac{23}{64}} \tag{3.28}$$

- **Determinazione del numero di bit d:**

$$d = \left\lceil \log_2 \frac{2}{s} \right\rceil = \left\lceil \log_2 \frac{2}{\frac{1}{32}} \right\rceil = \lceil \log_2 64 \rceil = \lceil 6 \rceil = \mathbf{6}\ \text{bit} \tag{3.29}$$

La chiamata finale alla procedura di conversione risulta quindi:

$$\text{CONVERTER}\left(\frac{23}{64},\ 6\right) \tag{3.30}$$

3.4.3 Dimostrazione di Correttezza del Troncamento a d Bit

Affinché il decodificatore possa ricostruire il messaggio originario senza ambiguità, il valore troncato a d bit $\hat{x}$ emesso dal convertitore deve cadere **rigorosamente all'interno dell'intervallo target**:

$$\hat{x} \in [l, l + s) \tag{3.31}$$

Dimostrazione. Si valuti l'errore massimo di troncamento 2^{-d}, dove $d = \lceil \log_2(2/s) \rceil$:

$$2^{-\lceil \log_2 \frac{2}{s} \rceil} = \frac{1}{2^{\lceil \log_2 \frac{2}{s} \rceil}} \leq \frac{1}{2^{\log_2 \frac{2}{s}}} = \frac{1}{\frac{2}{s}} = \frac{s}{2} \tag{3.32}$$

La disuguaglianza deriva dalla monotonia dell'esponenziale di base 2: poiché $\lceil y \rceil \geq y$, passando al reciproco vale $2^{-\lceil y \rceil} \leq 2^{-y}$. In virtù del Lemma 3.2.1, l'errore di troncamento soddisfa:

$$x - \hat{x} \leq 2^{-d} \leq \frac{s}{2} \tag{3.33}$$

Poiché $\hat{x}$ è ottenuto azzerando cifre binarie, vale sempre $\hat{x} \leq x$. Inoltre, $x = l + s/2$ è il punto medio esatto dell'intervallo, distante $s/2$ da entrambi gli estremi. Combinando le disuguaglianze:

$$l = x - \frac{s}{2} \leq \hat{x} \leq x < l + s \tag{3.34}$$

Ne consegue che:

$$\hat{x} \in [l, l + s) \tag{3.35}$$

garantendo che qualsiasi sequenza decodificata a partire da $\hat{x}$ corrisponda univocamente al testo originale T. □

3.4.4 Conversione Binaria Esplicita di $\frac{23}{64}$ in 6 Bit

La generazione effettiva dei bit può essere ricavata attraverso due modalità equivalenti:

Metodo 1: Scomposizione per Somma di Potenze di Due Si esprime il numeratore 23 come somma di potenze intere decrescenti di 2:

$$23 = 16 + 4 + 2 + 1 = 2^4 + 2^2 + 2^1 + 2^0 \tag{3.36}$$

Dividendo l'espressione per il denominatore $64 = 2^6$:

$$\frac{23}{64} = \frac{16}{64} + \frac{4}{64} + \frac{2}{64} + \frac{1}{64} = 2^{-2} + 2^{-4} + 2^{-5} + 2^{-6} \tag{3.37}$$

Mappando ciascun peso frazionario sui 6 bit di output:

- Peso $2^{-1} = 1/2$: non presente $\longrightarrow$ **0**
- Peso $2^{-2} = 1/4$: presente $(16/64)$ $\longrightarrow$ **1**
- Peso $2^{-3} = 1/8$: non presente $\longrightarrow$ **0**

- Peso $2^{-4} = 1/16$: presente $(4/64) \longrightarrow \mathbf{1}$
- Peso $2^{-5} = 1/32$: presente $(2/64) \longrightarrow \mathbf{1}$
- Peso $2^{-6} = 1/64$: presente $(1/64) \longrightarrow \mathbf{1}$

La rappresentazione binaria frazionaria è dunque:

$$\frac{23}{64} = [0.010111]_2 \implies \mathbf{b = 010111} \tag{3.38}$$

Metodo 2: Esecuzione Passo-Passo di CONVERTER L'applicazione formale dell'algoritmo di moltiplicazione per 2 produce la medesima sequenza (Tabella 3.1):

Tabella 3.1: Tracciamento iterativo delle moltiplicazioni per 2 eseguite da `CONVERTER`$(23/64, 6)$.

Passo j	**Ingresso** x_j	**Raddoppio** $y_j = 2x_j$	**Bit Estratto** $b_j = \lfloor y_j \rfloor$	**Residuo** $x_{j+1} = y_j - b_j$
1	23/64	$46/64 < 1$	**0**	46/64
2	46/64	$92/64 = 1 + 28/64$	**1**	28/64
3	28/64	$56/64 < 1$	**0**	56/64
4	56/64	$112/64 = 1 + 48/64$	**1**	48/64
5	48/64	$96/64 = 1 + 32/64$	**1**	32/64
6	32/64	$64/64 = 1 + 0$	**1**	0

La parola compressa emessa per il testo $T = abc$ è la sequenza binaria a 6 bit:

$$c(abc) = \mathbf{010111} \tag{3.39}$$

3.5 Teorema sull'Efficienza della Codifica Aritmetica

Il pregio cardine della codifica aritmetica risiede nell'efficienza asintotica: il sovraccosto rispetto all'entropia teorica di Shannon non dipende dalla lunghezza del testo m, ma è limitato da una costante prefissata.

Teorema 3.5.1 (Efficienza dell'Arithmetic Coding)**.** *Sia T un testo di m caratteri generato da una sorgente stazionaria priva di memoria con entropia $H(S)$. La dimensione totale in bit dell'output compresso generato dalla codifica aritmetica soddisfa rigorosamente:*

$$\textit{Dimensione Output} < m \cdot H(S) + 2 \textit{ bit} \tag{3.40}$$

Dimostrazione. La dimostrazione procede analizzando analiticamente la dimensione in bit L emessa dalla procedura di codifica:

1. **Espressione del numero di bit emessi:**
   La formula di emissione richiede una precisione pari a:
   $$L = \left\lceil \log_2 \frac{2}{s} \right\rceil \tag{3.41}$$

2. **Espansione algebrica del logaritmo:**
   Sfruttando le proprietà fondamentali dei logaritmi ($\log_2(2/s) = \log_2 2 - \log_2 s = 1 - \log_2 s$):
   $$L = \lceil 1 - \log_2 s \rceil \tag{3.42}$$

3. **Maggiorazione della funzione ceiling:**
   Per ogni numero reale y, vale la maggiorazione stretta $\lceil y \rceil < y + 1$. Applicandola a $y = 1 - \log_2 s$:
   $$L < (1 - \log_2 s) + 1 = 2 - \log_2 s \tag{3.43}$$

4. **Scomposizione dell'ampiezza finale s:**
   L'ampiezza dell'intervallo finale s per una sequenza di m caratteri $T[1 \dots m]$ è pari al prodotto delle probabilità dei singoli simboli incontrati:
   $$s = \prod_{i=1}^{m} p(T[i]) = \prod_{\sigma \in A} p(\sigma)^{\#\mathrm{occ}(\sigma)} \tag{3.44}$$
   dove $\#\mathrm{occ}(\sigma)$ denota il numero di occorrenze del simbolo σ nel testo T.

5. **Applicazione del logaritmo al prodotto:**
   Sostituendo l'espressione di s nella disuguaglianza:
   $$L < 2 - \log_2 \left(\prod_{\sigma \in A} p(\sigma)^{\#\mathrm{occ}(\sigma)} \right) = 2 - \sum_{\sigma \in A} \log_2 \left(p(\sigma)^{\#\mathrm{occ}(\sigma)} \right) \tag{3.45}$$
   Estraendo l'esponente dalla funzione logaritmo:
   $$L < 2 - \sum_{\sigma \in A} \#\mathrm{occ}(\sigma) \log_2 p(\sigma) \tag{3.46}$$

6. **Introduzione dell'entropia empirica:**
   Ricordando che $-\log_2 p(\sigma) = \log_2 \dfrac{1}{p(\sigma)}$, e moltiplicando e dividendo la sommatoria per la lunghezza complessiva m:
   $$L < 2 + m \sum_{\sigma \in A} \frac{\#\mathrm{occ}(\sigma)}{m} \log_2 \frac{1}{p(\sigma)} \tag{3.47}$$

Sotto l'ipotesi standard in cui le frequenze empiriche relative nel testo approssimano la distribuzione della sorgente ($\frac{\#\text{occ}(\sigma)}{m} \approx p(\sigma)$):

$$L < 2 + m \sum_{\sigma \in A} p(\sigma) \log_2 \frac{1}{p(\sigma)} \tag{3.48}$$

7. **Chiusura della dimostrazione:**
   Riconoscendo nella sommatoria la definizione classica dell'entropia di Shannon $H(S) = \sum_{\sigma \in A} p(\sigma) \log_2 \frac{1}{p(\sigma)}$ [1]:

$$L < m \cdot H(S) + 2 \text{ bit} \tag{3.49}$$

□

3.6 Analisi Comparativa: Codifica Aritmetica vs Huffman

Il confronto sistematico tra l'algoritmo di Huffman e la codifica aritmetica sintetizza l'evoluzione concettuale della compressione dati statistica (Tabella 3.2 e Figura 3.2).

Tabella 3.2: Quadro comparativo dettagliato tra Codifica di Huffman e Codifica Aritmetica.

Proprietà Analizzata	**Codifica di Huffman** [3]	**Codifica Aritmetica** [7, 8]
Principio Operativo	Mappatura simbolo per simbolo	Mappatura globale dell'intero testo
Granularità del Codice	Numero intero di bit per simbolo	Frazioni continue di bit per simbolo
Limite Superiore Dimensione	$< m \cdot H(S) + m$ bit	$< m \cdot H(S) + 2$ bit
Overhead Massimo Totale	Fino a $+m$ bit (+1 bit/simbolo)	Al più +2 bit per l'intero file
Tasso Medio per Simbolo (L/m)	$< H(S) + 1$ bit/simbolo	$< H(S) + \frac{2}{m} \xrightarrow{m \to \infty} H(S)$
Sorgenti Sbilanciate ($H \to 0$)	Prestazioni scarse (≥ 1 bit/simbolo)	Ottime (l'overhead di 2 bit svanisce per grandi m)
Complessità di Calcolo	Molto bassa (lookup e bitwise)	Media/Alta (aritmetica a precisione arbitraria)

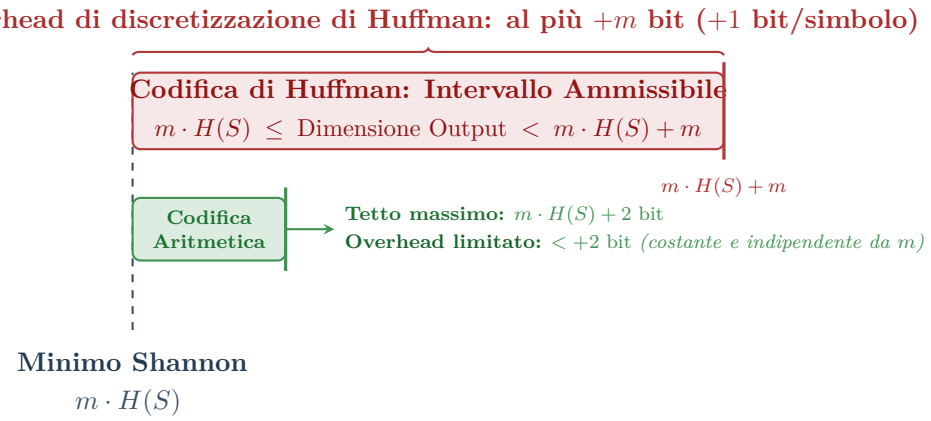


Figura 3.2: Confronto grafico dei massimi teorici di dimensione: mentre la codifica di Huffman accumula un overhead di discretizzazione proporzionale a m (fino a $+m$ bit), la codifica aritmetica confina l'errore a una quantità fissa e costante ($< +2$ bit), convergendo esattamente all'entropia per $m \to \infty$.

3.6.1 Dinamica dell'Ampiezza s e Bit Emessi

La dinamica dell'ampiezza residua $s = \prod_{i=1}^{m} p(T[i])$ fornisce un'intuizione geometrica immediata sul consumo di bit della codifica aritmetica:

- **Se $s \approx 1$:** tale scenario occorre quando i simboli elaborati possiedono una probabilità prossima a 1 (simboli estremamente frequenti). L'intervallo subisce una contrazione trascurabile, e la formula di emissione richiede una quantità minima di bit (*"few bits will be added to $l + s/2$"*);
- **Se $s \to 0$:** tale situazione si manifesta con l'aumentare della lunghezza m della sequenza o all'inclusione di caratteri rari. L'intervallo collassa rapidamente, richiedendo un numero proporzionato di bit per discriminare con esattezza il punto medio all'interno della partizione (*"lots of bits"*).

Mentre l'algoritmo di Huffman deve necessariamente sacrificare almeno un bit intero per ogni carattere del messaggio anche quando $p(\sigma) = 0.999$, la codifica aritmetica assegna idealmente frazioni di bit commisurate all'autoinformazione reale, eliminando la ridondanza strutturale della discretizzazione.

Capitolo 4

Esercitazioni di Compressione Statistica: Huffman e Codifica Aritmetica

4.1 Introduzione e Struttura della Sessione di Esercizi

La presente sessione di esercitazioni è dedicata all'applicazione pratica, analitica e implementativa dei due paradigmi fondamentali della compressione dati statistica senza perdita (*lossless*):

1. **La Codifica di Huffman** [3], affrontata sia nella sua costruzione classica basata su albero binario (*greedy bottom-up*), sia nella sua rappresentazione avanzata in **Forma Canonica** [4, 5], analizzando in dettaglio le strutture vettoriali compresse e l'algoritmo di decodifica privo di albero (*tree-less decoding*).

2. **La Codifica Aritmetica** [7, 8], esplorata mediante la suddivisione ricorsiva dello spazio di probabilità continuo unitario $[0, 1)$ e l'algoritmo di decodifica a intervalli annidati (*zoom-in decoding*), con particolare attenzione alla gestione rigorosa dei valori di frontiera negli intervalli semiaperti.

Ciascun esercizio viene sviluppato attraverso la formulazione teorica del modello di sorgente, la derivazione algebrica puntuale di tutti i passaggi di calcolo, la sintesi tabellare dei parametri intermedi e la visualizzazione geometrica mediante diagrammi dedicati.

4.2 Esercizio 1: Codifica di Huffman Standard e Analisi Topologica

4.2.1 Definizione della Sorgente e Distribuzione di Probabilità

Si consideri una sorgente discreta stazionaria e priva di memoria (*Discrete Memoryless Source*, DMS) definita su un alfabeto finito composto da $n = 5$ simboli:

$$A = \{a, b, c, d, e\} \tag{4.1}$$

A ciascun simbolo $\sigma \in A$ è associata una probabilità di emissione a priori $p(\sigma)$ definita dal seguente vettore di probabilità:

$$p(a) = 0.10 \tag{4.2}$$
$$p(b) = 0.14 \tag{4.3}$$
$$p(c) = 0.38 \tag{4.4}$$
$$p(d) = 0.18 \tag{4.5}$$
$$p(e) = 0.20 \tag{4.6}$$

Si verifica preliminarmente la condizione di normalizzazione della misura di probabilità sullo spazio campionario discreto:

$$\sum_{\sigma \in A} p(\sigma) = 0.10 + 0.14 + 0.38 + 0.18 + 0.20 = 1.00 \tag{4.7}$$

4.2.2 Costruzione Passo-Passo dell'Albero Greedy

L'algoritmo di Huffman opera aggregando iterativamente dal basso verso l'alto (*bottom-up*) le due radici associate ai pesi di probabilità minimi presenti nella foresta attiva di nodi.

1. **Stato Iniziale (Foresta F_0):**
   L'insieme iniziale dei nodi foglia, ordinati in senso debolmente crescente di probabilità, è dato da:

   $$F_0 = \{a : 0.10, \quad b : 0.14, \quad d : 0.18, \quad e : 0.20, \quad c : 0.38\} \tag{4.8}$$

2. **Passo 1 (Fusione di a e b):**
   I due elementi a probabilità minima assoluta sono a (0.10) e b (0.14).

Essi vengono estratti e fusi sotto un nuovo nodo composito intermedio (ab) avente peso aggregato:

$$p(ab) = p(a) + p(b) = 0.10 + 0.14 = 0.24 \tag{4.9}$$

La foresta attiva aggiornata assume la configurazione:

$$F_1 = \{d : 0.18, \quad e : 0.20, \quad (ab) : 0.24, \quad c : 0.38\} \tag{4.10}$$

3. **Passo 2 (Fusione di d ed e):**
   I due minimi correnti nella foresta F_1 sono le foglie d (0.18) ed e (0.20). La loro aggregazione genera un secondo nodo intermedio (de) avente peso cumulato:

$$p(de) = p(d) + p(e) = 0.18 + 0.20 = 0.38 \tag{4.11}$$

   La foresta attiva diviene:

$$F_2 = \{(ab) : 0.24, \quad (de) : 0.38, \quad c : 0.38\} \tag{4.12}$$

4. **Passo 3 (Fusione dei sottoalberi (ab) e (de)):**
   I due nodi a peso strettamente minore in F_2 sono (ab) (0.24) e (de) (0.38)[1]. La loro unione genera il nodo intermedio:

$$p(abde) = p(ab) + p(de) = 0.24 + 0.38 = 0.62 \tag{4.13}$$

   La foresta attiva comprende ora soltanto due radici:

$$F_3 = \{c : 0.38, \quad (abde) : 0.62\} \tag{4.14}$$

5. **Passo 4 (Fusione Finale verso la Radice):**
   L'ultima iterazione unisce la foglia isolata c (0.38) con il macro-nodo $(abde)$ (0.62), producendo la radice dell'albero di Huffman con peso normalizzato:

$$p(\text{root}) = p(c) + p(abde) = 0.38 + 0.62 = 1.00 \tag{4.15}$$

[1]Si osservi che sia (de) sia la foglia c possiedono peso pari a 0.38. La scelta di fondere (ab) con (de) oppure con c costituisce uno spareggio arbitrario privo di impatto sulla lunghezza media globale del codice, ma produce alberi topologicamente distinti. In accordo con la traccia della lezione, si procede aggregando (ab) con (de).

4.2.3 Topologia dell'Albero e Assegnamento delle Codeword

Fissata la convenzione standard che assegna il bit `0` ad ogni diramazione sinistra (*left branch*) e il bit `1` ad ogni diramazione destra (*right branch*), la struttura topologica risultante è illustrata in Figura 4.1.

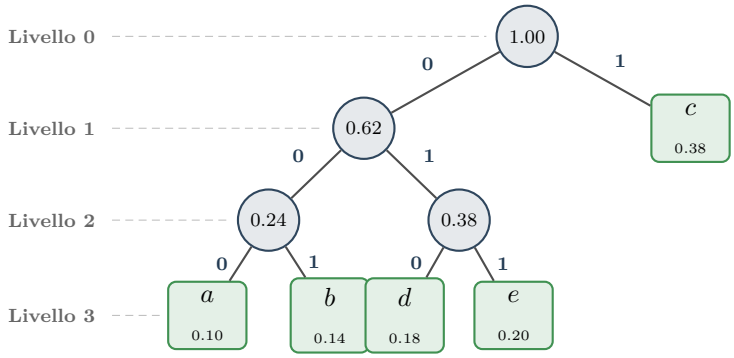


Figura 4.1: Albero di Huffman generato per l'Esercizio 1. I nodi circolari blu rappresentano le radici e le fusioni intermedie con la probabilità aggregata; i rettangoli verdi indicano le foglie associate ai caratteri della sorgente.

Seguendo il percorso orientato dalla radice a ogni foglia, si ottiene l'insieme delle parole di codice standard riportate in Tabella 4.1.

Tabella 4.1: Insieme delle parole di codice generate dall'albero di Huffman standard per l'alfabeto A.

Simbolo σ	**Probabilità** $p(\sigma)$	**Percorso Topologico**	**Codeword Standard**	**Lunghezza** $\ell(\sigma)$
c	0.38	Destra	`1`	1 bit
a	0.10	Sinistra → Sinistra → Sinistra	`000`	3 bit
b	0.14	Sinistra → Sinistra → Destra	`001`	3 bit
d	0.18	Sinistra → Destra → Sinistra	`010`	3 bit
e	0.20	Sinistra → Destra → Destra	`011`	3 bit

4.2.4 Proprietà Strutturali e Lunghezza Media

Dall'analisi della topologia dell'albero emergono rilevanti proprietà analitiche:

- **Asimmetria strutturale marcata:** Il simbolo c, caratterizzato dalla probabilità dominante ($p(c) = 0.38$), viene isolato come foglia diretta della radice al livello 1 ($\ell(c) = 1$). Gli altri quattro simboli $\{a, b, d, e\}$, possedendo probabilità individuali sensibilmente inferiori, sono confinati al livello 3 ($\ell = 3$).

- **Assenza di foglie a profondità intermedia:** Non vi sono simboli associati a parole di lunghezza 2 ($\ell = 2$). Il livello 2 è popolato esclusivamente da nodi intermedi di raccordo, ossia (ab) e (de).

La lunghezza media attesa del codice è pari a:

$$\begin{aligned} L(C) &= \sum_{\sigma \in A} p(\sigma) \cdot \ell(\sigma) \\ &= 0.38 \cdot 1 + (0.10 + 0.14 + 0.18 + 0.20) \cdot 3 \\ &= 0.38 + 0.62 \cdot 3 = 0.38 + 1.86 = 2.24 \text{ bit/simbolo} \end{aligned} \tag{4.16}$$

Confrontando tale valore con l'entropia di Shannon $H(S)$ della sorgente:

$$\begin{aligned} H(S) &= -\sum_{\sigma \in A} p(\sigma) \log_2 p(\sigma) \\ &= -\Big(0.10 \log_2 0.10 + 0.14 \log_2 0.14 + 0.38 \log_2 0.38 \\ &\quad + 0.18 \log_2 0.18 + 0.20 \log_2 0.20\Big) \\ &\approx 2.1855 \text{ bit/simbolo} \end{aligned} \tag{4.17}$$

La ridondanza media del codice risulta estremamente contenuta:

$$R = L(C) - H(S) \approx 2.24 - 2.1855 = 0.0545 \text{ bit/simbolo} \tag{4.18}$$

rispettando pienamente il teorema di Shannon $H(S) \leq L(C) < H(S) + 1$.

4.3 Esercizio 1: Costruzione della Codifica di Huffman Canonica

4.3.1 Vettori Descrittori NUM e SYMB

Richiamando la teoria dei codici canonici introdotta in sezione 2.7, la standardizzazione delle parole preserva la distribuzione ottimale delle lunghezze $\{\ell(\sigma)\}_{\sigma \in A}$ dell'albero di Huffman (Figura 4.1), imponendo l'ordinamento lessicografico compatto per eliminare la memorizzazione esplicita dei puntatori.

Dalla topologia dell'albero, la profondità massima è $\ell_{\max} = 3$. Si istanziano i vettori descrittori:

- **Vettore NUM delle lunghezze** (conteggio delle foglie per $\ell \in \{1, 2, 3\}$):

$$\text{NUM} = [\text{NUM}[1], \text{NUM}[2], \text{NUM}[3]] = [1, 0, 4] \tag{4.19}$$

- **Vettore SYMB dei simboli** (ordinamento alfabetico per livello):

$$\text{SYMB} = \begin{cases} \text{Livello } 1: & [c] \\ \text{Livello } 2: & \emptyset \\ \text{Livello } 3: & [a, b, d, e] \end{cases} \tag{4.20}$$

4.3.2 Derivazione Ricorsiva del Vettore First Code (FC)

I codici base $FC[\ell]$ vengono determinati mediante la ricorrenza all'indietro introdotta in sezione 2.8:

$$FC[\ell_{\max}] = 0, \qquad FC[\ell - 1] = \left\lfloor \frac{FC[\ell] + \text{NUM}[\ell]}{2} \right\rfloor \tag{4.21}$$

Applicando la relazione ai parametri dell'esercizio ($\ell_{\max} = 3$):

1. Per il livello massimo $\ell = 3$:

$$FC[3] = 0 \tag{4.22}$$

2. Per il livello intermedio $\ell = 2$:

$$FC[2] = \left\lfloor \frac{FC[3] + \text{NUM}[3]}{2} \right\rfloor = \left\lfloor \frac{0 + 4}{2} \right\rfloor = \lfloor 2.0 \rfloor = 2 \tag{4.23}$$

3. Per il livello minimo $\ell = 1$:

$$FC[1] = \left\lfloor \frac{FC[2] + \text{NUM}[2]}{2} \right\rfloor = \left\lfloor \frac{2 + 0}{2} \right\rfloor = \lfloor 1.0 \rfloor = 1 \tag{4.24}$$

Il vettore risultante dei codici base di partenza è dunque:

$$FC = [FC[1], FC[2], FC[3]] = [1, 2, 0] \tag{4.25}$$

4.3.3 Assegnamento delle Parole di Codice Canoniche

Fissato il vettore FC, la parola di codice per il generico simbolo σ di lunghezza ℓ, avente indice ordinale $i \in \{0, 1, \ldots, \text{NUM}[\ell] - 1\}$ all'interno del sottovettore $\text{SYMB}[\ell]$, si ottiene semplicemente sommando l'offset i al valore di base ed esprimendo il risultato in notazione binaria su esattamente ℓ bit:

$$C_{\text{canonico}}(\sigma) = \text{bin}_\ell(FC[\ell] + i) \tag{4.26}$$

Procedendo per ciascuna lunghezza:

- **Livello $\ell = 1$ ($FC[1] = 1$):**
  - Simbolo c ($i = 0$):

$$\text{Valore} = 1 + 0 = 1 \implies \text{Codeword} = \texttt{1} \tag{4.27}$$

- **Livello $\ell = 2$ ($FC[2] = 2$):**
  - Nessun simbolo presente (NUM[2] = 0).
- **Livello $\ell = 3$ ($FC[3] = 0$):**
  - Simbolo a ($i = 0$):

$$\text{Valore} = 0 + 0 = 0 \implies \text{Codeword} = \texttt{000} \tag{4.28}$$

  - Simbolo b ($i = 1$):

$$\text{Valore} = 0 + 1 = 1 \implies \text{Codeword} = \texttt{001} \tag{4.29}$$

  - Simbolo d ($i = 2$):

$$\text{Valore} = 0 + 2 = 2 \implies \text{Codeword} = \texttt{010} \tag{4.30}$$

  - Simbolo e ($i = 3$):

$$\text{Valore} = 0 + 3 = 3 \implies \text{Codeword} = \texttt{011} \tag{4.31}$$

In Tabella 4.2 si riporta il confronto sinottico tra il codice standard ricavato dall'albero greedy e il codice canonico formalizzato.

Tabella 4.2: Confronto analitico tra codici di Huffman standard e canonici ricavati per l'alfabeto A.

Simbolo σ	**Probabilità $p(\sigma)$**	**Lunghezza ℓ**	**Codeword Standard**	**Codeword Canonica**
c	0.38	1	1	1
a	0.10	3	000	000
b	0.14	3	001	001
d	0.18	3	010	010
e	0.20	3	011	011

Nota 4.3.1. Nel caso specifico dell'Esercizio 1, la convenzione di ordinamento 0/1 adottata nell'albero greedy coincide numericamente con l'output della forma canonica. Tale coincidenza riflette l'ordinamento naturale imposto alle fusioni, ma in scenari generali la forma canonica riordina le parole all'interno di ciascun livello per garantire l'ordine lessicografico rigoroso sui simboli sorgente.

4.4 Esercizio 1: Decodifica Senza Albero (*Tree-less Decoding*)

4.4.1 Meccanismo del Registro Intero e Ricerca Tabellare

Come analizzato in sezione 2.9, la decodifica canonica non richiede l'instanziazione dell'albero binario, ma opera mediante l'aggiornamento iterativo del registro intero $v \leftarrow 2v + b$ e il confronto con la soglia $FC[\ell]$ del livello corrente. Se $v \geq FC[\ell]$, la parola viene identificata con offset index $= v - FC[\ell]$ nel vettore SYMB$[\ell]$.

L'assenza di puntatori riduce l'occupazione di memoria ai soli tre compatti array unidimensionali (NUM, SYMB, FC), eliminando integralmente le letture casuali nella cache tipiche della visita su trie.

4.4.2 Traccia Analitica di Decodifica su Sequenza di Bit

Si consideri il seguente flusso compresso binario ricevuto dal decodificatore:

$$\mathcal{B} = \texttt{1 0 0 0 1 0 1 1 0 1 1 1 0 0} \tag{4.32}$$

Si richiede di tracciare la decodifica analitica dei primi tre simboli della sequenza, documentando le variazioni del registro v e i confronti con il vettore $FC = [1, 2, 0]$.

Decodifica del Simbolo 1

- Si resetta il registro: $v \leftarrow 0$.
- **Livello** $\ell = 1$:
  - Si estrae il primo bit dal flusso: $b = \texttt{1}$.
  - Aggiornamento del registro:

$$v = 2 \cdot 0 + 1 = 1 \tag{4.33}$$

  - Test di confronto con $FC[1] = 1$:

$$v = 1 \geq FC[1] = 1 \quad \Longrightarrow \quad \textbf{Condizione Verificata!} \tag{4.34}$$

  - Calcolo dell'offset all'interno del livello 1:

$$\text{index} = v - FC[1] = 1 - 1 = 0 \tag{4.35}$$

- Identificazione del simbolo nel vettore SYMB[1]:

$$\sigma_1 = \text{SYMB}[1][0] = \mathbf{c} \tag{4.36}$$

- Bit consumati: 1 (`'1'`). Output emesso: **c**.

Decodifica del Simbolo 2

- Si resetta il registro: $v \leftarrow 0$.
- **Livello** $\ell = 1$:
  - Si estrae il bit successivo: $b = \texttt{0}$.
  - Aggiornamento del registro: $v = 2 \cdot 0 + 0 = 0$.
  - Test di confronto con $FC[1] = 1$:

$$v = 0 \geq FC[1] = 1 \quad \Longrightarrow \quad \text{Falso. Si avanza a } \ell = 2. \tag{4.37}$$

- **Livello** $\ell = 2$:
  - Si estrae il bit successivo: $b = \texttt{0}$.
  - Aggiornamento del registro: $v = 2 \cdot 0 + 0 = 0$.
  - Test di confronto con $FC[2] = 2$:

$$v = 0 \geq FC[2] = 2 \quad \Longrightarrow \quad \text{Falso. Si avanza a } \ell = 3. \tag{4.38}$$

- **Livello** $\ell = 3$:
  - Si estrae il bit successivo: $b = \texttt{0}$.
  - Aggiornamento del registro: $v = 2 \cdot 0 + 0 = 0$.
  - Test di confronto con $FC[3] = 0$:

$$v = 0 \geq FC[3] = 0 \quad \Longrightarrow \quad \textbf{Condizione Verificata!} \tag{4.39}$$

  - Calcolo dell'offset all'interno del livello 3:

$$\text{index} = v - FC[3] = 0 - 0 = 0 \tag{4.40}$$

  - Identificazione del simbolo nel vettore SYMB[3]:

$$\sigma_2 = \text{SYMB}[3][0] = \mathbf{a} \tag{4.41}$$

- Bit consumati: 3 (`'000'`). Output emesso: **a**.

Decodifica del Simbolo 3

- Si resetta il registro: $v \leftarrow 0$.
- **Livello** $\ell = 1$:
  - Si estrae il bit successivo: $b = 1$.
  - Aggiornamento del registro: $v = 2 \cdot 0 + 1 = 1$.
  - Test di confronto con $FC[1] = 1$:

$$v = 1 \geq FC[1] = 1 \quad \Longrightarrow \quad \textbf{Condizione Verificata!} \tag{4.42}$$

  - Calcolo dell'offset:

$$\text{index} = v - FC[1] = 1 - 1 = 0 \implies \sigma_3 = \text{SYMB}[1][0] = \mathbf{c} \tag{4.43}$$

- Bit consumati: 1 (`'1'`). Output emesso: **c**.

I primi tre caratteri estratti dalla sequenza sono pertanto:

$$\mathbf{c}, \quad \mathbf{a}, \quad \mathbf{c} \tag{4.44}$$

In Tabella 4.3 si riassume l'evoluzione temporale degli stati del decodificatore per i tre simboli elaborati.

Tabella 4.3: Traccia tabellare dell'algoritmo di decodifica senza albero per i primi tre simboli del flusso $\mathcal{B}$.

Simbolo	**Livello** ℓ	**Bit Letto** b	**Registro** v	**Soglia** $FC[\ell]$	**Condizione** $v \geq FC[\ell]$	**Simbolo Emesso**
1	1	1	1	1	Vero (idx = 0)	**c**
2	1	0	0	1	Falso	–
2	2	0	0	2	Falso	–
2	3	0	0	0	Vero (idx = 0)	**a**
3	1	1	1	1	Vero (idx = 0)	**c**

Nota 4.4.1 (Decodifica dell'intero bitstream)**.** Continuando l'elaborazione sui bit rimanenti della stringa $\mathcal{B} = \underline{1}\ \underline{000}\ \underline{1}\ 011\ 011\ 1\ 00$, si osserva:

- Bit $6, 7, 8$ (`011`): livello $1 \to v = 0 < 1$; livello $2 \to v = 1 < 2$; livello $3 \to v = 3 \geq FC[3] = 0$, idx $= 3 - 0 = 3 \implies$ **e**.
- Bit $9, 10, 11$ (`011`): parimenti decodificato nel simbolo **e**.
- Bit 12 (`1`): decodificato nel simbolo **c**.

La sequenza complessiva decodificata corrisponde a **c a c e e c** $\ldots$ confermando l'assoluta integrità del prefisso e l'istantaneità della decodifica.

4.5 Esercizio 2: Codifica Aritmetica – Modello di Sorgente e Partizioni

4.5.1 Definizione dell'Alfabeto Ternario e Punti di Ancoraggio

Si consideri una sorgente discreta definita su un alfabeto ternario $A = \{a, b, c\}$, le cui probabilità di emissione a priori sono assegnate in forma razionale esatta:

$$p(a) = \frac{1}{2} = 0.50, \qquad p(b) = \frac{1}{4} = 0.25, \qquad p(c) = \frac{1}{4} = 0.25 \tag{4.45}$$

Si verifica immediatamente:

$$\sum_{\sigma \in A} p(\sigma) = \frac{1}{2} + \frac{1}{4} + \frac{1}{4} = 1.00 \tag{4.46}$$

Fissato l'ordinamento naturale dell'alfabeto $a < b < c$, i punti cumulativi di ancoraggio (*cumulative distribution function*, F) sul dominio unitario $[0, 1)$ sono formalizzati da:

$$F(a) = 0 \tag{4.47}$$

$$F(b) = p(a) = \frac{1}{2} \tag{4.48}$$

$$F(c) = p(a) + p(b) = \frac{1}{2} + \frac{1}{4} = \frac{3}{4} \tag{4.49}$$

$$F(\text{end}) = F(c) + p(c) = \frac{3}{4} + \frac{1}{4} = 1 \tag{4.50}$$

4.5.2 Regola di Partizione Ricorsiva degli Intervalli Semiaperti

Dato un generico intervallo semiaperto a destra $[l, l + s)$ avente base $l \in [0, 1)$ e ampiezza $s \in (0, 1]$, la comparsa di ciascun simbolo dell'alfabeto A restringe lo spazio al sotto-intervallo definito dalle seguenti equazioni analitiche:

$$I(a) = [l + s \cdot F(a),\ l + s \cdot F(b)) = \left[l,\ l + \frac{1}{2}s\right) \tag{4.51}$$

$$I(b) = [l + s \cdot F(b),\ l + s \cdot F(c)) = \left[l + \frac{1}{2}s,\ l + \frac{3}{4}s\right) \tag{4.52}$$

$$I(c) = [l + s \cdot F(c),\ l + s \cdot F(\text{end})) = \left[l + \frac{3}{4}s,\ l + s\right) \tag{4.53}$$

Come esposto in sezione 3.1, ciascun sotto-intervallo $I(\sigma) = [l_\sigma, r_\sigma)$ è chiuso a sinistra e aperto a destra. Tale convenzione topologica garantisce che il punto di confine $r_{\sigma_1} = l_{\sigma_2}$ appartenga univocamente a $I(\sigma_2)$ e non a $I(\sigma_1)$, scongiurando ogni ambiguità durante la localizzazione numerica (aspetto critico evidenziato nell'Iterazione 3 al valore di soglia $r_a = l_b = 46/64$, sezione 4.6.2).

4.6 Esercizio 2: Decodifica Aritmetica Passo-Passo Mediante Zoom-In

4.6.1 Dati Ricevuti e Calcolo del Valore Razionale

Il messaggio compresso trasmesso al decodificatore è costituito dalla coppia ordinata:

$$\langle C, m \rangle = \langle 10111_2,\ 4 \rangle \tag{4.54}$$

dove:

- $C = 10111_2$ rappresenta la sequenza binaria di $k = 5$ bit estratta dalla procedura `CONVERTER`;
- $m = 4$ indica la lunghezza esatta della sequenza di caratteri da ricostruire.

Il valore frazionario reale $x \in [0, 1)$ associato alla stringa binaria C è espresso dalla somma delle potenze negative di due:

$$\begin{aligned} x &= \sum_{j=1}^{5} b_j \cdot 2^{-j} = \frac{1}{2^1} + \frac{0}{2^2} + \frac{1}{2^3} + \frac{1}{2^4} + \frac{1}{2^5} \\ &= \frac{16}{32} + \frac{0}{32} + \frac{4}{32} + \frac{2}{32} + \frac{1}{32} = \frac{23}{32} = 0.71875 \end{aligned} \tag{4.55}$$

Il processo di decodifica opera mantenendo invariato il punto target $x = \frac{23}{32}$ e restringendo progressivamente per $m = 4$ volte l'intervallo attivo, individuando iterativamente quale sotto-intervallo contiene x.

4.6.2 Derivazione Dettagliata delle Iterazioni di Zoom-In

Iterazione 1 ($k = 1$)

- Intervallo iniziale: $[l_0, r_0) = [0, 1)$, ampiezza $s_0 = 1$.

- Partizione dello spazio in 32-esimi:

$$I(a) = \left[0,\ \frac{1}{2}\right) = \left[\frac{0}{32},\ \frac{16}{32}\right) \tag{4.56}$$

$$I(b) = \left[\frac{1}{2},\ \frac{3}{4}\right) = \left[\frac{16}{32},\ \frac{24}{32}\right) \tag{4.57}$$

$$I(c) = \left[\frac{3}{4},\ 1\right) = \left[\frac{24}{32},\ \frac{32}{32}\right) \tag{4.58}$$

- Collocazione del valore target $x = \frac{23}{32}$:

$$\frac{16}{32} \leq \frac{23}{32} < \frac{24}{32} \iff x \in I(b) \tag{4.59}$$

- **1° Simbolo Decodificato: b**.

- Nuovo intervallo per l'iterazione 2:

$$[l_1, r_1) = \left[\frac{16}{32},\ \frac{24}{32}\right), \quad s_1 = \frac{24}{32} - \frac{16}{32} = \frac{8}{32} = \frac{1}{4} \tag{4.60}$$

Iterazione 2 ($k = 2$)

- Intervallo corrente: $[l_1, r_1) = \left[\frac{16}{32},\ \frac{24}{32}\right)$, con $s_1 = \frac{8}{32}$.

- Calcolo dei sotto-intervalli:

$$I(a) = \left[\frac{16}{32},\ \frac{16}{32} + \frac{1}{2} \cdot \frac{8}{32}\right) = \left[\frac{16}{32},\ \frac{20}{32}\right) \tag{4.61}$$

$$I(b) = \left[\frac{20}{32},\ \frac{20}{32} + \frac{1}{4} \cdot \frac{8}{32}\right) = \left[\frac{20}{32},\ \frac{22}{32}\right) \tag{4.62}$$

$$I(c) = \left[\frac{22}{32},\ \frac{22}{32} + \frac{1}{4} \cdot \frac{8}{32}\right) = \left[\frac{22}{32},\ \frac{24}{32}\right) \tag{4.63}$$

- Collocazione del valore target $x = \frac{23}{32}$:

$$\frac{22}{32} \leq \frac{23}{32} < \frac{24}{32} \iff x \in I(c) \tag{4.64}$$

- **2° Simbolo Decodificato: c**.

- Nuovo intervallo per l'iterazione 3:

$$[l_2, r_2) = \left[\frac{22}{32},\ \frac{24}{32}\right), \quad s_2 = \frac{2}{32} = \frac{1}{16} \tag{4.65}$$

Iterazione 3 ($k = 3$) – Gestione Critica del Punto di Frontiera

- Intervallo corrente: $[l_2, r_2) = \left[\frac{22}{32}, \frac{24}{32}\right)$, ampiezza $s_2 = \frac{2}{32} = \frac{4}{64}$.
- Per operare con frazioni a denominatore comune, esprimiamo l'intervallo e il valore target in 64-esimi:

$$[l_2, r_2) = \left[\frac{44}{64}, \frac{48}{64}\right), \qquad x = \frac{23}{32} = \frac{46}{64} \tag{4.66}$$

- Calcolo analitico dei tre sotto-intervalli:

$$I(a) = \left[\frac{44}{64}, \frac{44}{64} + \frac{1}{2} \cdot \frac{4}{64}\right) = \left[\frac{44}{64}, \frac{46}{64}\right) \tag{4.67}$$

$$I(b) = \left[\frac{46}{64}, \frac{46}{64} + \frac{1}{4} \cdot \frac{4}{64}\right) = \left[\frac{46}{64}, \frac{47}{64}\right) \tag{4.68}$$

$$I(c) = \left[\frac{47}{64}, \frac{47}{64} + \frac{1}{4} \cdot \frac{4}{64}\right) = \left[\frac{47}{64}, \frac{48}{64}\right) \tag{4.69}$$

- **Analisi rigorosa della frontiera su $x = \frac{46}{64}$:**
  - Il sotto-intervallo $I(a) = \left[\frac{44}{64}, \frac{46}{64}\right)$ è strettamente aperto a destra:

$$y \in I(a) \iff \frac{44}{64} \leq y < \frac{46}{64} \implies \frac{46}{64} \notin I(a) \tag{4.70}$$

  - Il sotto-intervallo $I(b) = \left[\frac{46}{64}, \frac{47}{64}\right)$ è chiuso a sinistra:

$$y \in I(b) \iff \frac{46}{64} \leq y < \frac{47}{64} \implies \frac{46}{64} \in I(b) \tag{4.71}$$

  Poiché la disuguaglianza $\frac{46}{64} \leq x < \frac{47}{64}$ è rigorosamente verificata, il punto target appartiene unicamente al sotto-intervallo associato al simbolo b.

- **3° Simbolo Decodificato: b**.
- Nuovo intervallo per l'iterazione 4:

$$[l_3, r_3) = \left[\frac{46}{64}, \frac{47}{64}\right), \quad s_3 = \frac{1}{64} = \frac{2}{128} \tag{4.72}$$

Iterazione 4 ($k = 4$)

- Intervallo corrente scalato in 128-esimi:
$$[l_3, r_3) = \left[\frac{92}{128}, \frac{94}{128}\right), \quad s_3 = \frac{2}{128} \tag{4.73}$$
- Il valore target in 128-esimi coincide esattamente con la base dell'intervallo:
$$x = \frac{46}{64} = \frac{92}{128} \tag{4.74}$$
- Suddivisione nei sotto-intervalli:
$$I(a) = \left[\frac{92}{128}, \frac{92}{128} + \frac{1}{2} \cdot \frac{2}{128}\right) = \left[\frac{92}{128}, \frac{93}{128}\right) \tag{4.75}$$
$$I(b) = \left[\frac{93}{128}, \frac{93}{128} + \frac{1}{4} \cdot \frac{2}{128}\right) = \left[\frac{93}{128}, \frac{93.5}{128}\right) = \left[\frac{186}{256}, \frac{187}{256}\right) \tag{4.76}$$
$$I(c) = \left[\frac{187}{256}, \frac{188}{256}\right) \tag{4.77}$$
- Collocazione del valore target $x = \frac{92}{128}$:
$$\frac{92}{128} \leq x = \frac{92}{128} < \frac{93}{128} \iff x \in I(a) \tag{4.78}$$
- **4º Simbolo Decodificato: a**.
- **Condizione di Terminazione:** Raggiunto il numero prefissato di simboli $m = 4$. L'algoritmo arresta la scansione.

La sequenza complessiva decodificata dalla coppia $\langle 10111_2, 4\rangle$ è pertanto:

$$\textbf{b c b a} \tag{4.79}$$

4.6.3 Riepilogo Tabellare e Diagramma Geometrico di Zoom-In

In Tabella 4.4 viene documentata la successione completa dei parametri analitici per ciascun passo di decodifica.

Tabella 4.4: Quadro sinottico delle quattro iterazioni di zoom-in per la decodifica aritmetica del valore $x = 23/32$.

Passo k	Base l_{k-1}	Ampiezza s_{k-1}	Rappr. Frazionaria	Valore Target x	Intervallo di Caduta	Simbolo
1	0	1	$[0/32, 32/32)$	$23/32 = 0.71875$	$[16/32, 24/32)$	**b**
2	1/2	1/4	$[16/32, 24/32)$	$23/32 = 0.71875$	$[22/32, 24/32)$	**c**
3	11/16	1/16	$[44/64, 48/64)$	$46/64 = 0.71875$	$[46/64, 47/64)$	**b**
4	23/32	1/64	$[92/128, 94/128)$	$92/128 = 0.71875$	$[92/128, 93/128)$	**a**

In Figura 4.2 è visualizzato il restringimento progressivo dei sotto-intervalli e la posizione invariante del punto $x = 0.71875$.

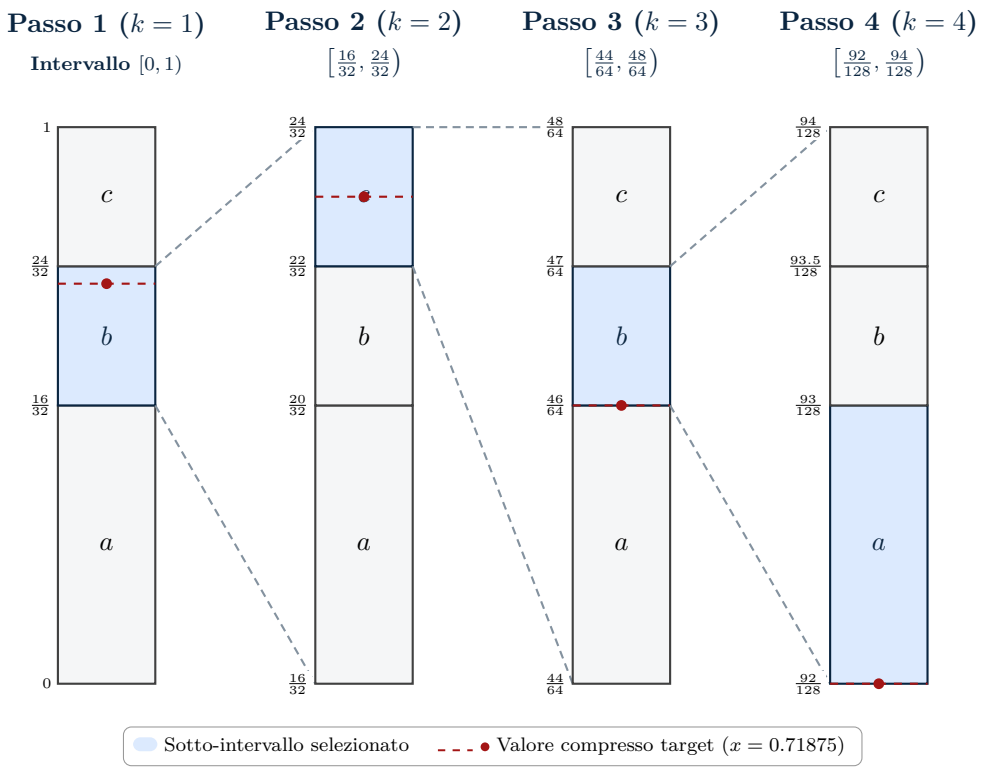


Figura 4.2: Visualizzazione geometrica dei quattro stadi di decodifica aritmetica progressiva (*zoom-in*) per il valore compresso $x = \frac{23}{32} = 0.71875$. I rettangoli azzurri denotano i sotto-intervalli di volta in volta selezionati, mentre la retta tratteggiata rossa identifica la posizione invariante del punto.

4.7 Quadro Comparativo e Considerazioni Metodologiche

La risoluzione analitica dei due esercizi evidenzia la profonda divergenza metodologica tra i compressori statistici a livello di singolo simbolo e i codificatori d'intervallo continui. In Tabella 4.5 viene delineato un confronto sistematico tra i due approcci.

Tabella 4.5: Quadro sinottico comparativo tra la codifica di Huffman canonica (Esercizio 1) e la codifica aritmetica (Esercizio 2).

Dimensione Analitica	**Codifica di Huffman Canonica**	**Codifica Aritmetica**
Natura dell'Assegnamento	Discreta: ogni simbolo riceve un numero intero $\ell \geq 1$ di bit.	Continua: l'intero messaggio mappa in un sotto-intervallo $[l, l+s)$.
Struttura Decodificatore	Array compatti NUM, SYMB, FC; decodifica *tree-less* via registro intero.	Aritmetica razionale/virgola fissa; restringimento ricorsivo per zoom-in.
Complessità di Decodifica	$O(1)$ operazioni elementari (shift e confronti) per bit letto.	$O(1)$ moltiplicazioni/sottrazioni a precisione finita per simbolo estratto.
Condizioni di Frontiera	Assenti: il prefisso univoco impedisce sovrapposizioni.	Critiche: rigorosa appartenenza all'estremo chiuso sinistro $[l_\sigma, r_\sigma)$.
Overhead di Lunghezza	Limitato da $L(C) < H(S) + 1$, sensibile a simboli con $p(\sigma) > 0.5$.	Strettamente asintotico: overhead totale < 2 bit per l'intera sequenza.

Nota 4.7.1 (Trattamento delle Frontiere e Precisione Finita)**.** L'iterazione 3 dell'Esercizio 2 costituisce un paradigma didattico di primaria importanza per l'ingegneria degli algoritmi: qualora si adotti un confronto errato del tipo $y \leq r$ anziché $y < r$, il valore $x = 46/64$ verrebbe erroneamente attribuito al simbolo a, alterando l'intero albero di decodifica a valle. L'assoluta coerenza della convenzione semiaperta a destra $[l, r)$ garantisce la natura priva di ambiguità (*injective mapping*) del processo di decompressione.

Capitolo 5

Compressione Statistica Avanzata: Codifica Aritmetica Dinamica, Huffman Canonico e Codici Ottimi Non-Huffman

5.1 Quadro Generale della Sessione di Esercizi

La presente sessione di approfondimento avanzato esplora i limiti teorici e i confini applicativi dei compressori statistici senza perdita (*lossless*), focalizzandosi su due tematiche cardine dell'ingegneria degli algoritmi:

1. **La Codifica Aritmetica Adattiva (Modello Dinamico):** Il superamento dell'ipotesi classica di stazionarietà e conoscenza preventiva della distribuzione di probabilità [7, 8]. Quando le frequenze dei simboli non sono note a priori, il sistema stima e aggiorna i conteggi in modo incrementale e sincrono durante la scansione del testo, impiegando lo stimatore bayesiano uniforme di Laplace per prevenire il problema delle frequenze nulle.

2. **Huffman Canonico e Teoria dell'Ottimalità dei Codici a Prefisso:** La costruzione rigorosa della forma canonica per distribuzioni sbilanciate (*unbalanced*), l'analisi dell'efficienza rispetto all'entropia di Shannon [1], e la trattazione formale del problema dell'ottimalità dei codici prefissi. In particolare, si dimostra che l'algoritmo greedy di Huffman [3] rappresenta una condizione **sufficiente ma non necessaria** per l'ottimalità, provando l'esistenza di schemi di codifica prefissi ottimi non generabili dall'algoritmo di Huffman.

5.2 Codifica Aritmetica Adattiva a Modello Dinamico

5.2.1 Formulazione del Problema e Limiti del Modello Statico

Nei modelli statistici semi-statici (sezione 1.8 e sezione 2.5), il codificatore effettua un passaggio preliminare sul testo per determinarne la distribuzione globale $\{p(\sigma)\}_{\sigma \in A}$, trasmettendola nell'header con relativo overhead in bit. Nel **modello dinamico o adattivo**, invece, la distribuzione a priori non è nota e non viene trasmesso alcun dizionario statico: codificatore e decodificatore mantengono una struttura di conteggio sincronizzata, aggiornando le frequenze cumulative online dopo ogni simbolo elaborato.

L'obiettivo dell'esercizio è la codifica della sequenza di $m = 4$ simboli:

$$T = b\, b\, a\, c \tag{5.1}$$

definita sull'alfabeto ternario finito $A = \{a, b, c\}$.

5.2.2 Inizializzazione dei Conteggi Mediante Stimatore di Laplace

Se all'avvio dell'algoritmo si assegnassero conteggi nulli ai simboli non ancora osservati, la loro probabilità risulterebbe pari a zero, provocando un'ampiezza dell'intervallo nulla ($s = 0$) che bloccherebbe irreparabilmente il processo di codifica alla prima occorrenza di un nuovo carattere.

Per garantire la convergenza e la correttezza della suddivisione dello spazio reale unitario, si adotta la classica **correzione bayesiana uniforme** (*stimatore di Laplace* o *add-one smoothing*), inizializzando a 1 il contatore di ciascun simbolo dell'alfabeto:

- Conteggi iniziali ($k = 0$):

$$C_0(a) = 1, \qquad C_0(b) = 1, \qquad C_0(c) = 1 \tag{5.2}$$

- Totale iniziale delle osservazioni virtuali:

$$N_0 = \sum_{\sigma \in A} C_0(\sigma) = 1 + 1 + 1 = 3 \tag{5.3}$$

- Probabilità condizionate assegnate al primo simbolo ($k = 1$):

$$p_1(a) = \frac{C_0(a)}{N_0} = \frac{1}{3}, \qquad p_1(b) = \frac{C_0(b)}{N_0} = \frac{1}{3}, \qquad p_1(c) = \frac{C_0(c)}{N_0} = \frac{1}{3} \tag{5.4}$$

Lo spazio di partenza coincide con l'intervallo reale unitario semiaperto:

$$[l_0, l_0 + s_0) = [0, 1), \qquad \text{con } l_0 = 0, \quad s_0 = 1 \tag{5.5}$$

5.2.3 Derivazione Passo-Passo delle Quattro Iterazioni

Passo 1: Elaborazione del simbolo $T[1] = b$

- **Distribuzione e intervalli locali al passo 1 ($N_0 = 3$):**

$$I_1(a) = \left[0,\ \frac{1}{3}\right), \quad p_1(a) = \frac{1}{3} \tag{5.6}$$

$$I_1(b) = \left[\frac{1}{3},\ \frac{2}{3}\right), \quad p_1(b) = \frac{1}{3} \tag{5.7}$$

$$I_1(c) = \left[\frac{2}{3},\ 1\right), \quad p_1(c) = \frac{1}{3} \tag{5.8}$$

- **Aggiornamento dell'intervallo globale:**

$$l_1 = l_0 + s_0 \cdot \frac{1}{3} = 0 + 1 \cdot \frac{1}{3} = \frac{1}{3} \tag{5.9}$$

$$s_1 = s_0 \cdot p_1(b) = 1 \cdot \frac{1}{3} = \frac{1}{3} \tag{5.10}$$

  Intervallo globale risultante:

$$[l_1, l_1 + s_1) = \left[\frac{1}{3},\ \frac{2}{3}\right) \tag{5.11}$$

- **Aggiornamento dei conteggi:** Si incrementa il contatore del simbolo b:

$$C_1(b) = C_0(b) + 1 = 1 + 1 = 2 \implies N_1 = 1 + 2 + 1 = 4 \tag{5.12}$$

Passo 2: Elaborazione del simbolo $T[2] = b$

- **Distribuzione e intervalli locali al passo 2 ($N_1 = 4$):**

$$p_2(a) = \frac{1}{4} \implies I_2(a) = \left[0,\ \frac{1}{4}\right) \tag{5.13}$$

$$p_2(b) = \frac{2}{4} = \frac{1}{2} \implies I_2(b) = \left[\frac{1}{4},\ \frac{3}{4}\right) \tag{5.14}$$

$$p_2(c) = \frac{1}{4} \implies I_2(c) = \left[\frac{3}{4},\ 1\right) \tag{5.15}$$

- **Aggiornamento dell'intervallo globale:**

$$l_2 = l_1 + s_1 \cdot \frac{1}{4} = \frac{1}{3} + \frac{1}{3} \cdot \frac{1}{4} = \frac{1}{3} + \frac{1}{12} = \frac{4+1}{12} = \frac{5}{12} \tag{5.16}$$

$$s_2 = s_1 \cdot p_2(b) = \frac{1}{3} \cdot \frac{2}{4} = \frac{1}{6} = \frac{2}{12} \tag{5.17}$$

Intervallo globale risultante:

$$[l_2, l_2 + s_2) = \left[\frac{5}{12},\ \frac{5}{12} + \frac{2}{12}\right) = \left[\frac{5}{12},\ \frac{7}{12}\right) \tag{5.18}$$

- **Aggiornamento dei conteggi:** Si incrementa nuovamente il contatore di b:

$$C_2(b) = C_1(b) + 1 = 2 + 1 = 3 \implies N_2 = 1 + 3 + 1 = 5 \tag{5.19}$$

Passo 3: Elaborazione del simbolo $T[3] = a$

- **Distribuzione e intervalli locali al passo 3 ($N_2 = 5$):**

$$p_3(a) = \frac{1}{5} \implies I_3(a) = \left[0,\ \frac{1}{5}\right) \tag{5.20}$$

$$p_3(b) = \frac{3}{5} \implies I_3(b) = \left[\frac{1}{5},\ \frac{4}{5}\right) \tag{5.21}$$

$$p_3(c) = \frac{1}{5} \implies I_3(c) = \left[\frac{4}{5},\ 1\right) \tag{5.22}$$

- **Aggiornamento dell'intervallo globale:**

$$l_3 = l_2 + s_2 \cdot 0 = \frac{5}{12} \tag{5.23}$$

$$s_3 = s_2 \cdot p_3(a) = \frac{1}{6} \cdot \frac{1}{5} = \frac{1}{30} \tag{5.24}$$

Esprimendo base e ampiezza sul minimo comune denominatore 60:

$$l_3 = \frac{5 \cdot 5}{12 \cdot 5} = \frac{25}{60}, \qquad s_3 = \frac{1 \cdot 2}{30 \cdot 2} = \frac{2}{60} \tag{5.25}$$

Intervallo globale risultante:

$$[l_3, l_3 + s_3) = \left[\frac{25}{60},\ \frac{27}{60}\right) \tag{5.26}$$

- **Aggiornamento dei conteggi:** Si registra la comparsa del simbolo a:

$$C_3(a) = C_2(a) + 1 = 1 + 1 = 2 \implies N_3 = 2 + 3 + 1 = 6 \tag{5.27}$$

Passo 4: Elaborazione del simbolo $T[4] = c$

- **Distribuzione e intervalli locali al passo 4 ($N_3 = 6$):**

$$p_4(a) = \frac{2}{6} = \frac{1}{3} \tag{5.28}$$

$$p_4(b) = \frac{3}{6} = \frac{1}{2} \tag{5.29}$$

$$p_4(c) = \frac{1}{6} \tag{5.30}$$

L'estremo inferiore locale del simbolo c è definito dalla somma delle probabilità dei simboli che lo precedono:

$$F_4(c) = \sum_{\tau<c} p_4(\tau) = p_4(a) + p_4(b) = \frac{2}{6} + \frac{3}{6} = \frac{5}{6} \tag{5.31}$$

L'intervallo locale associato a c è pertanto:

$$I_4(c) = \left[\frac{5}{6},\ 1\right) \tag{5.32}$$

- **Aggiornamento dell'intervallo globale:**

$$\begin{aligned} l_4 &= l_3 + s_3 \cdot \frac{5}{6} = \frac{25}{60} + \frac{1}{30} \cdot \frac{5}{6} = \frac{25}{60} + \frac{5}{180} \\ &= \frac{75}{180} + \frac{5}{180} = \frac{80}{180} \end{aligned} \tag{5.33}$$

$$s_4 = s_3 \cdot p_4(c) = \frac{1}{30} \cdot \frac{1}{6} = \frac{1}{180} \tag{5.34}$$

- **Rappresentazione finale su base 360:** Per calcolare agevolmente il punto centrale e semplificare la conversione binaria, si estendono base e ampiezza al denominatore 360:

$$l_4 = \frac{80 \cdot 2}{180 \cdot 2} = \frac{160}{360}, \qquad s_4 = \frac{1 \cdot 2}{180 \cdot 2} = \frac{2}{360} \tag{5.35}$$

L'intervallo compresso definitivo che mappa biunivocamente la stringa $T = bbac$ è:

$$\text{COMPRESS}(bbac) = \left[\frac{160}{360},\ \frac{162}{360}\right) \tag{5.36}$$

5.2.4 Rappresentazione Geometrica delle Fasi di Compressione

In Figura 5.1 è illustrata l'evoluzione grafica delle suddivisioni dell'intervallo a modello dinamico attraverso i quattro passi di codifica.

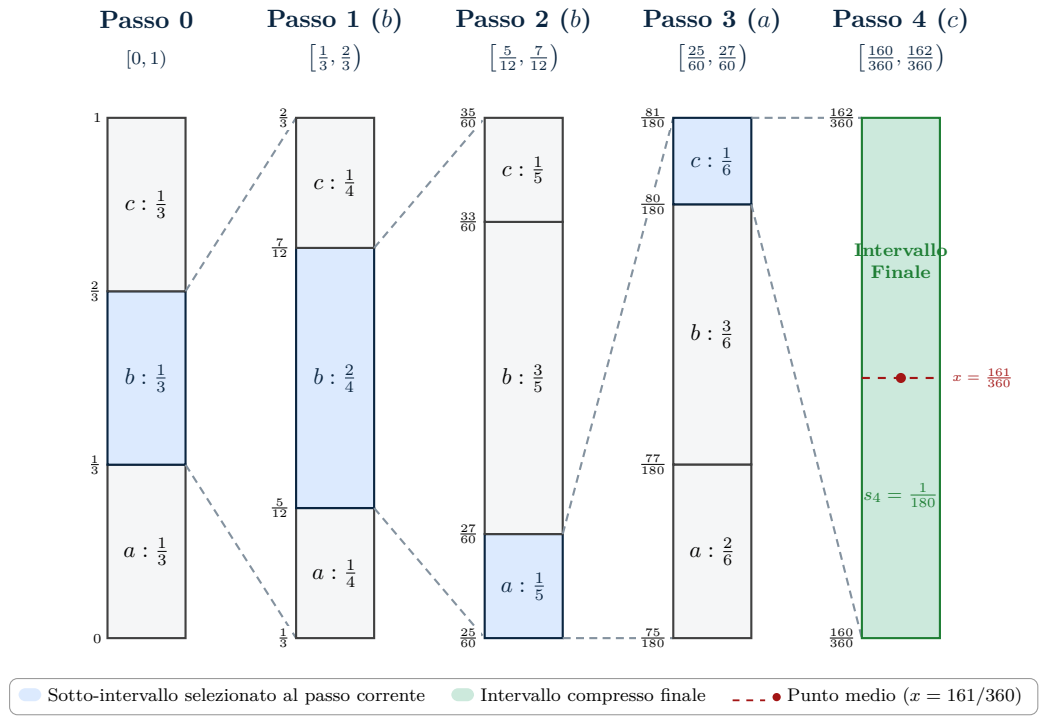


Figura 5.1: Suddivisione progressiva dello spazio unitario nella codifica aritmetica adattiva per il messaggio $T = bbac$. I rettangoli azzurri indicano i sotto-intervalli selezionati in base ai simboli via via acquisiti, evidenziando la variazione dinamica delle ampiezze proporzionali ai conteggi di Laplace.

5.2.5 Calcolo dei Parametri di Output: Lunghezza in Bit e Punto Medio

In virtù del criterio di precisione e del lemma di troncamento dimostrati in sezione 3.4, la rappresentazione frazionaria non ambigua dell'intervallo finale richiede un numero di bit pari a:

$$\#\text{Bits} = \left\lceil \log_2 \frac{2}{s_4} \right\rceil = \lceil \log_2(360) \rceil = 9 \text{ bit} \tag{5.37}$$

poiché $2^8 = 256 < 360 \leq 512 = 2^9$.

La coordinata reale interna selezionata per la conversione è il punto medio esatto dell'intervallo finale $[l_4, l_4 + s_4)$:

$$x = l_4 + \frac{s_4}{2} = \frac{160}{360} + \frac{1}{360} = \frac{161}{360} \approx 0.447222 \tag{5.38}$$

5.2.6 Esecuzione Passo-Passo dell'Algoritmo `CONVERTER`

Applicando la procedura `CONVERTER` formalizzata in sottosezione 3.1.2, la frazione razionale $x = \frac{161}{360}$ viene convertita nella parola binaria di $k = 9$ bit mediante moltiplicazioni successive per 2 ed estrazione della parte intera:

1. **Iterazione 1** $(j = 1)$**:**

$$y_1 = 2 \cdot \frac{161}{360} = \frac{161}{180} < 1 \implies \mathbf{b_1 = 0}, \quad \text{residuo: } x_2 = \frac{161}{180} \tag{5.39}$$

2. **Iterazione 2** $(j = 2)$**:**

$$y_2 = 2 \cdot \frac{161}{180} = \frac{161}{90} \geq 1 \implies \mathbf{b_2 = 1}, \quad \text{residuo: } x_3 = \frac{161}{90} - 1 = \frac{71}{90} \tag{5.40}$$

3. **Iterazione 3** $(j = 3)$**:**

$$y_3 = 2 \cdot \frac{71}{90} = \frac{71}{45} \geq 1 \implies \mathbf{b_3 = 1}, \quad \text{residuo: } x_4 = \frac{71}{45} - 1 = \frac{26}{45} \tag{5.41}$$

4. **Iterazione 4** $(j = 4)$**:**

$$y_4 = 2 \cdot \frac{26}{45} = \frac{52}{45} \geq 1 \implies \mathbf{b_4 = 1}, \quad \text{residuo: } x_5 = \frac{52}{45} - 1 = \frac{7}{45} \tag{5.42}$$

5. **Iterazione 5** $(j = 5)$**:**

$$y_5 = 2 \cdot \frac{7}{45} = \frac{14}{45} < 1 \implies \mathbf{b_5 = 0}, \quad \text{residuo: } x_6 = \frac{14}{45} \tag{5.43}$$

6. **Iterazione 6** $(j = 6)$**:**

$$y_6 = 2 \cdot \frac{14}{45} = \frac{28}{45} < 1 \implies \mathbf{b_6 = 0}, \quad \text{residuo: } x_7 = \frac{28}{45} \tag{5.44}$$

7. **Iterazione 7** $(j = 7)$**:**

$$y_7 = 2 \cdot \frac{28}{45} = \frac{56}{45} \geq 1 \implies \mathbf{b_7 = 1}, \quad \text{residuo: } x_8 = \frac{56}{45} - 1 = \frac{11}{45} \tag{5.45}$$

8. **Iterazione 8 ($j = 8$):**

$$y_8 = 2 \cdot \frac{11}{45} = \frac{22}{45} < 1 \implies \mathbf{b_8 = 0}, \quad \text{residuo: } x_9 = \frac{22}{45} \tag{5.46}$$

9. **Iterazione 9 ($j = 9$):**

$$y_9 = 2 \cdot \frac{22}{45} = \frac{44}{45} < 1 \implies \mathbf{b_9 = 0}, \quad \text{arresto (raggiunti } k = 9 \text{ bit)} \tag{5.47}$$

La parola di codice compressa emessa è:

$$C(bbac) = \texttt{011100100} \tag{5.48}$$

In Tabella 5.1 si sintetizzano i passaggi della procedura `CONVERTER`.

Tabella 5.1: Traccia computazionale dell'algoritmo `CONVERTER` per il valore $x = 161/360$ su 9 bit di precisione.

Iterazione j	**Valore Corrente** x_j	**Raddoppio** $y_j = 2x_j$	**Bit Generato** $b_j = \lfloor y_j \rfloor$	**Residuo Frazionario** x_{j+1}	**Valore Decimale Approssimato**
1	161/360	$161/180 \approx 0.8944$	**0**	161/180	0.0_2
2	161/180	$161/90 \approx 1.7889$	**1**	71/90	0.01_2
3	71/90	$71/45 \approx 1.5778$	**1**	26/45	0.011_2
4	26/45	$52/45 \approx 1.1556$	**1**	7/45	0.0111_2
5	7/45	$14/45 \approx 0.3111$	**0**	14/45	0.01110_2
6	14/45	$28/45 \approx 0.6222$	**0**	28/45	0.011100_2
7	28/45	$56/45 \approx 1.2444$	**1**	11/45	0.0111001_2
8	11/45	$22/45 \approx 0.4889$	**0**	22/45	0.01110010_2
9	22/45	$44/45 \approx 0.9778$	**0**	44/45	0.011100100_2

5.2.7 Formato Finale dell'Output e Header di Terminazione

Il messaggio finale compresso trasmesso al canale di comunicazione viene formalizzato dalla coppia ordinata:

$$\langle \texttt{011100100},\ 4 \rangle \tag{5.49}$$

Nota 5.2.1 (Condizione di Arresto e Parametro di Lunghezza)**.** Come approfondito in sezione 3.4, la codifica aritmetica individua un punto nel continuo reale senza delimitatori intrinseci di parola. Il decodificatore per zoom-in necessita pertanto di una condizione d'arresto deterministica, fornita alternativamente dall'esplicitazione della lunghezza della sequenza ($m = 4$) nell'header oppure dall'emissione di uno speciale simbolo di terminazione dedicato (EOF).

5.3 Huffman Canonico con Distribuzione Sbilanciata

5.3.1 Definizione della Sorgente e Normalizzazione

Si consideri una sorgente discreta definita su un alfabeto composto da $n = 6$ simboli:

$$A = \{a, b, c, d, e, f\} \tag{5.50}$$

avente una distribuzione di probabilità fortemente polarizzata:

$$p(a) = p(b) = p(c) = p(d) = 0.10 \tag{5.51}$$

$$p(e) = p(f) = 0.30 \tag{5.52}$$

Verifica della misura di probabilità:

$$\sum_{\sigma \in A} p(\sigma) = 4 \cdot 0.10 + 2 \cdot 0.30 = 0.40 + 0.60 = 1.00 \tag{5.53}$$

5.3.2 Topologia dell'Albero Canonico con Foglie a Sinistra

In conformità ai principi costruttivi della forma canonica esposti in sezione 2.7, le foglie dell'albero vengono raggruppate per profondità e collocate a sinistra:

- Le 4 foglie dei simboli a probabilità minore $\{a, b, c, d\}$ risiedono alla profondità massima $\ell = 3$;
- Le 2 foglie dei simboli a probabilità maggiore $\{e, f\}$ risiedono alla profondità $\ell = 2$.

In Figura 5.2 è illustrata la topologia dell'albero binario risultante.

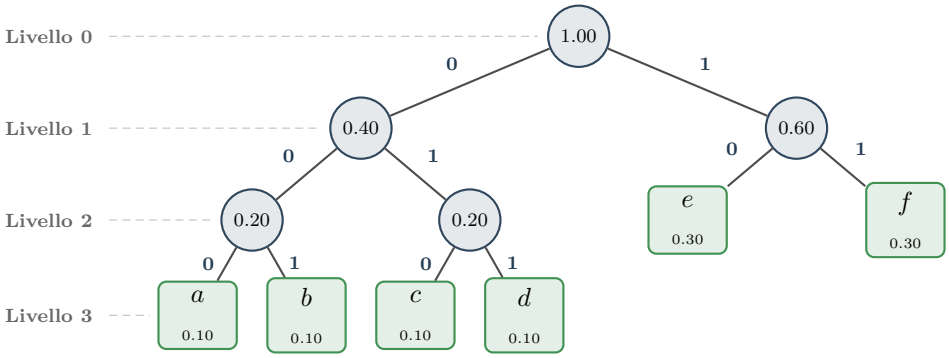


Figura 5.2: Albero binario di Huffman in forma canonica per la sorgente sbilanciata a 6 simboli. Tutte le foglie a profondità $\ell = 3$ sono aggregate a sinistra, mentre le foglie a profondità $\ell = 2$ occupano la porzione destra.

5.3.3 Vettori Descrittori NUM, SYMB e Ricorrenza FC

Dalla topologia canonica a profondità massima $\ell_{\max} = 3$ si ricavano i vettori descrittori introdotti in sezione 2.8:

- **Vettore NUM**$[\ell]$**:** $\mathrm{NUM} = [0, 2, 4]$;
- **Tabella SYMB**$[\ell]$ (ordinata lessicograficamente per ciascun livello): $\mathrm{SYMB}[1] \to \emptyset$, $\mathrm{SYMB}[2] \to [e, f]$, $\mathrm{SYMB}[3] \to [a, b, c, d]$;
- **Derivazione del vettore First Code (**$FC[\ell]$**):** Ponendo la condizione al livello fogliare massimo $FC[3] = 0$ (000_2) e applicando la ricorrenza all'indietro:

$$FC[\ell - 1] = \left\lfloor \frac{FC[\ell] + \mathrm{NUM}[\ell]}{2} \right\rfloor \tag{5.54}$$

si ricava:

$$FC[2] = \left\lfloor \frac{0 + 4}{2} \right\rfloor = 2 \quad (10_2) \tag{5.55}$$

$$FC[1] = \left\lfloor \frac{2 + 2}{2} \right\rfloor = 2 \tag{5.56}$$

da cui il vettore completo $FC = [2, 2, 0]$.

In Tabella 5.2 sono elencate le parole di codice canoniche generate per tutti i simboli della sorgente.

Tabella 5.2: Parole di codice canoniche assegnate all'alfabeto sbilanciato A.

Simbolo σ	**Probabilità** $p(\sigma)$	**Lunghezza** ℓ	**Offset in SYMB**$[\ell]$	**Valore Intero:** $FC[\ell] + \textbf{offset}$	**Codeword Canonica**
e	0.30	2	0	$2+0=2$	10
f	0.30	2	1	$2+1=3$	11
a	0.10	3	0	$0+0=0$	000
b	0.10	3	1	$0+1=1$	001
c	0.10	3	2	$0+2=2$	010
d	0.10	3	3	$0+3=3$	011

5.3.4 Lunghezza Media ed Efficienza Rispetto all'Entropia

Calcolo della Lunghezza Media $L(C)$

La lunghezza media attesa del codice canonico è data dalla media pesata delle lunghezze:

$$\begin{aligned} L(C) &= \sum_{\sigma \in A} p(\sigma) \cdot |c(\sigma)| \\ &= 4 \cdot (0.10 \cdot 3) + 2 \cdot (0.30 \cdot 2) \\ &= 4 \cdot 0.30 + 2 \cdot 0.60 = 1.20 + 1.20 = \mathbf{2.40} \text{ bit/simbolo} \end{aligned} \tag{5.57}$$

Calcolo dell'Entropia di Shannon $H(S)$

L'entropia teorica della sorgente S è espressa da:

$$\begin{aligned} H(S) &= \sum_{\sigma \in A} p(\sigma) \log_2 \frac{1}{p(\sigma)} \\ &= 4 \cdot \left(0.10 \log_2 \frac{1}{0.10} \right) + 2 \cdot \left(0.30 \log_2 \frac{1}{0.30} \right) \\ &= 0.40 \log_2(10) + 0.60 \log_2 \left(\frac{10}{3} \right) \end{aligned} \tag{5.58}$$

Poiché $\log_2(10) \approx 3.321928$ e $\log_2\left(\frac{10}{3}\right) = \log_2(10) - \log_2(3) \approx 3.321928 - 1.584963 = 1.736966$:

$$\begin{aligned} H(S) &\approx 0.40 \cdot 3.321928 + 0.60 \cdot 1.736966 \\ &= 1.328771 + 1.042179 = \mathbf{2.37095} \text{ bit/simbolo} \end{aligned} \tag{5.59}$$

Analisi della Ridondanza

La ridondanza associata al codice di Huffman canonico risulta pari a:

$$R = L(C) - H(S) \approx 2.40 - 2.37095 = 0.02905 \text{ bit/simbolo} \tag{5.60}$$

Poiché ciascun singolo simbolo sorgente deve essere tassativamente codificato con un numero intero di bit, il valore 2.40 bit/simbolo rappresenta il limite inferiore assoluto raggiungibile da qualunque schema di codifica a prefisso simbolo-per-simbolo.

5.4 Analisi Critica: Esistenza di Codici Ottimi Non Derivabili da Huffman

5.4.1 Proposta dello Schema Alternativo

Per verificare i fondamenti della teoria dei codici a prefisso, si consideri la seguente assegnazione alternativa di sequenze binarie definita sulla medesima sorgente:

$$a \longrightarrow \texttt{000} \quad (|c(a)| = 3) \tag{5.61}$$
$$b \longrightarrow \texttt{001} \quad (|c(b)| = 3) \tag{5.62}$$
$$e \longrightarrow \texttt{01} \quad (|c(e)| = 2) \tag{5.63}$$
$$c \longrightarrow \texttt{100} \quad (|c(c)| = 3) \tag{5.64}$$
$$d \longrightarrow \texttt{101} \quad (|c(d)| = 3) \tag{5.65}$$
$$f \longrightarrow \texttt{11} \quad (|c(f)| = 2) \tag{5.66}$$

L'analisi critica di tale codice si articola nella risoluzione analitica di quattro quesiti fondamentali.

5.4.2 Quesito 1: Il Codice è Privo di Prefisso (*Prefix-Free*)?

Proposizione 5.4.1. *Il codice alternativo descritto dalle equazioni* (5.61)–(5.66) *è rigorosamente privo di prefisso (prefix-free).*

Dimostrazione. Si consideri l'albero binario associato alla mappatura delle parole (illustrato in Figura 5.3). Ciascun simbolo sorgente $\sigma \in \{a, b, c, d, e, f\}$ corrisponde in modo biunivoco a un **nodo foglia** terminale dell'albero:

- Le foglie a, b, c, d risiedono a profondità 3 e non possiedono nodi discendenti;
- Le foglie e e f risiedono a profondità 2; i loro nodi omologhi a livello 2 non hanno rami uscenti verso ulteriori simboli, bensì rappresentano punti terminali del cammino.

Poiché nessun simbolo è collocato su un nodo interno dell'albero, nessuna parola di codice può costituire il prefisso iniziale di un'altra parola. Il codice garantisce pertanto la decodificabilità istantanea e univoca. □

5.4.3 Quesito 2: Il Codice è in Forma Canonica?

Proposizione 5.4.2. *Il codice alternativo non è in forma canonica.*

Dimostrazione. La definizione strutturale di codice canonico impone che le parole di codice siano disposte secondo un ordinamento lessicografico coerente con le rispettive lunghezze, confinando tutte le foglie a profondità maggiore verso un'unica estremità dell'albero (generalmente a sinistra). Nel codice proposto:

- Le parole che iniziano con il prefisso `00` ($a \to$ `000`, $b \to$ `001`) hanno lunghezza 3;
- La parola $e \to$ `01` ha lunghezza 2;
- Le parole successive che iniziano per `10` ($c \to$ `100`, $d \to$ `101`) hanno nuovamente lunghezza 3;
- La parola $f \to$ `11` ha lunghezza 2.

La presenza di parole più corte (`01` e `11`) intervallate e alternate a blocchi di parole più lunghe (`00x` e `10x`) viola radicalmente la sequenzialità compatta della forma canonica, impedendo la decodifica priva di albero mediante il semplice registro intero $v \geq FC[\ell]$. □

5.4.4 Quesito 3: Il Codice è Generabile dall'Algoritmo di Huffman?

Teorema 5.4.1. *L'albero binario associato al codice alternativo non può essere generato da alcuna sequenza di scelte ammissibili dell'algoritmo di Huffman.*

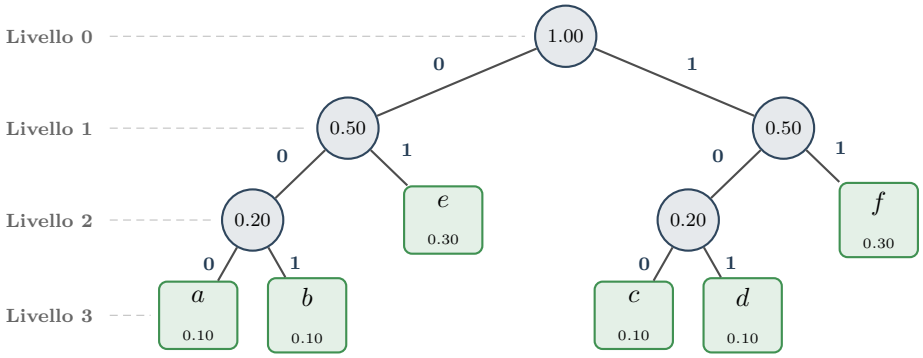


Figura 5.3: Topologia dell'albero associato al codice alternativo ottimo. I due rami principali hanno peso simmetrico 0.50 ciascuno, ma la loro costruzione interna viola il principio di scelta greedy di Huffman.

Dimostrazione. Si analizzi la traccia di esecuzione dell'algoritmo di Huffman a partire dai pesi della sorgente $F_0 = \{a : 0.1, b : 0.1, c : 0.1, d : 0.1, e : 0.3, f : 0.3\}$:

1. Al primo passo, l'algoritmo greedy individua i due nodi di peso minimo assoluto, ad esempio a (0.1) e b (0.1), aggregandoli nel nodo intermedio (ab) con peso:
$$p(ab) = 0.1 + 0.1 = 0.2 \tag{5.67}$$

2. Al secondo passo, i due nodi a peso minimo nella foresta attiva $\{c : 0.1, d : 0.1, (ab) : 0.2, e : 0.3, f : 0.3\}$ sono c (0.1) e d (0.1). La loro fusione genera il nodo intermedio (cd) con peso:
$$p(cd) = 0.1 + 0.1 = 0.2 \tag{5.68}$$

3. La foresta attiva al terzo passo comprende esattamente quattro nodi:
$$F_2 = \{(ab) : 0.20, \quad (cd) : 0.20, \quad e : 0.30, \quad f : 0.30\} \tag{5.69}$$

4. **Violazione del vincolo greedy di Huffman:** La regola fondamentale e invariante dell'algoritmo di Huffman impone tassativamente di fondere **i due nodi con le probabilità minime assolute**. Nella foresta F_2, i due pesi strettamente inferiori sono:
$$\min(F_2) = \{(ab) : 0.20,\ (cd) : 0.20\} \tag{5.70}$$
Pertanto, l'algoritmo di Huffman deve **obbligatoriamente** aggregare il nodo intermedio (ab) con il nodo intermedio (cd), generando un unico macro-nodo a quattro foglie $(abcd)$ avente peso $0.20 + 0.20 = 0.40$.

5. Al contrario, nella topologia dell'albero alternativo di Figura 5.3:

   - Il nodo (ab) (0.20) è stato fuso con la foglia e (0.30) per formare il ramo sinistro (abe) di peso 0.50;
   - Il nodo (cd) (0.20) è stato fuso con la foglia f (0.30) per formare il ramo destro (cdf) di peso 0.50.

   Tale scelta unisce una radice di peso 0.20 con una di peso 0.30, ignorando deliberatamente la presenza dell'altro nodo disponibile di peso 0.20.

Poiché la fusione effettuata contravviene in modo manifesto alla regola greedy, tale albero non può derivare da alcuna configurazione ammissibile dell'algoritmo di Huffman. □

5.4.5 Quesito 4: Il Codice è Ottimo?

Teorema 5.4.2 (Ottimalità Globale del Codice Alternativo)**.** *Il codice alternativo descritto dalle equazioni* (5.61)–(5.66) *è un codice a prefisso globale ottimo.*

Dimostrazione. Calcoliamo la lunghezza media ponderata attesa L_{alt} per lo schema alternativo:

$$\begin{aligned} L_{\text{alt}} &= \sum_{\sigma \in A} p(\sigma) \cdot |c_{\text{alt}}(\sigma)| \\ &= p(a)|c(a)| + p(b)|c(b)| + p(c)|c(c)| \\ &\quad + p(d)|c(d)| + p(e)|c(e)| + p(f)|c(f)| \\ &= 4 \cdot (0.10 \cdot 3) + 2 \cdot (0.30 \cdot 2) \\ &= 1.20 + 1.20 = \mathbf{2.40} \text{ bit/simbolo} \end{aligned} \tag{5.71}$$

Confrontando tale risultato con la lunghezza media del codice di Huffman L_{Huffman} calcolata nella sottosezione 5.3.4:

$$L_{\text{alt}} = L_{\text{Huffman}} = 2.40 \text{ bit/simbolo} \tag{5.72}$$

Poiché il teorema di Huffman stabilisce che nessun codice a prefisso su base simbolo-per-simbolo può possedere una lunghezza media strettamente inferiore a L_{Huffman}, e la lunghezza di L_{alt} coincide esattamente con tale minimo teorico, il codice alternativo è **strettamente ottimo**. □

5.4.6 Conseguenze Teoriche: Relazione tra Ottimalità e Algoritmo di Huffman

Questo risultato analitico conduce a un principio fondamentale della teoria dell'informazione e della progettazione degli algoritmi:

Corollario 5.4.1 (Sufficienza e Non-Necessità di Huffman)**.** *L'algoritmo di Huffman costituisce una condizione* ***sufficiente*** *per la determinazione di un codice prefisso ottimo con lunghezze intere, ma* ***non necessaria****:*

$$\textit{Codice Generato da Huffman} \implies \textit{Codice Ottimo} \tag{5.73}$$

$$\textit{Codice Ottimo} \not\implies \textit{Codice Generato da Huffman} \tag{5.74}$$

Lo spazio di tutti i codici a prefisso ottimi può contenere configurazioni topologiche alternative con distribuzioni identiche o compatibili di lunghezze di parola $\{\ell(\sigma)\}_{\sigma \in A}$ che non sono esplorate né generate dalla sequenza deterministica di fusioni minime prescritte dall'algoritmo greedy.

5.5 Quadro Comparativo dei Paradigmi Avanzati

In Tabella 5.3 si sintetizza il confronto comparativo tra i tre argomenti cardine approfonditi nella sessione.

Tabella 5.3: Quadro sinottico comparativo tra Codifica Aritmetica Adattiva, Huffman Canonico e Codici Ottimi Non-Huffman.

Aspetto Metodologico	**Codifica Aritmetica Adattiva**	**Huffman Canonico vs Non-Huffman**
Conoscenza a Priori	Assente: probabilità stimate dinamicamente tramite conteggi di Laplace.	Necessaria: probabilità statiche note prima della costruzione dell'albero.
Rappresentazione	Intervallo razionale continuo e punto medio convertito su 9 bit.	Albero binario discreto e vettori compatti $\mathrm{NUM}, \mathrm{SYMB}, FC$.
Condizione di Terminazione	Obbligatorio specificare la lunghezza $m = 4$ o un carattere EOF.	Implicita: la proprietà di prefisso individua istantaneamente ogni simbolo.
Ottimalità Computazionale	Overhead < 2 bit globale sull'intera sequenza, esente da troncamento a intero.	Ottimo rispetto al vincolo discreto intero (2.40 bit vs $H \approx 2.371$).
Spazio delle Soluzioni	Mappatura unica per la sequenza di conteggi e convenzione semiaperta.	Molteplicità di alberi ottimi; Huffman esplora solo un sottoinsieme greedy.

Nota 5.5.1 (Implicazioni di Ingegneria degli Algoritmi)**.** La comprensione che l'ottimalità prefissa non appartiene in modo esclusivo alle topologie generate da Huffman è di primaria importanza per i sistemi di compressione reali. Strutture come i codici canonici vincolano volutamente la topologia dell'albero per massimizzare la località dei dati nella cache ed eliminare i

puntatori, preservando rigorosamente la lunghezza media ottima calcolata da Shannon.

Bibliografia

[1] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

[2] Gonzalo Navarro. *Compact Data Structures: A Practical Approach*. Cambridge University Press, 2016.

[3] David A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101, 1952.

[4] Daniel S. Hirschberg and Debra A. Lelewer. Efficient decoding of prefix codes. *Communications of the ACM*, 33(4):449–459, 1990.

[5] Alistair Moffat and Andrew Turpin. On the implementation of minimum-redundancy prefix codes. *IEEE Transactions on Communications*, 45(10):1200–1207, 1997.

[6] Jarek Duda. Asymmetric numeral systems: Entropy coding combining speed of huffman coding with compression rate of arithmetic coding. *arXiv preprint arXiv:0902.0271*, 2009.

[7] Jorma Rissanen. Generalized kraft inequality and arithmetic coding. *IBM Journal of Research and Development*, 20(3):198–203, 1976.

[8] Ian H. Witten, Radford M. Neal, and John G. Cleary. Arithmetic coding for data compression. *Communications of the ACM*, 30(6):520–540, 1987.